

АНОТАЦІЯ

Верещак Б. О., Патерега Ю. І. (керівник). Розроблення системи динамічної адаптації складності рівнів у ігровому середовищі. – Національний університет «Львівська політехніка», Львів, 2026 р.

Розширена анотація

Роботу присвячено порівняльному аналізу методів динамічної адаптації складності рівнів (DDA) у комп'ютерних іграх. Дослідження зосереджено на аналізі ефективності алгоритмів на основі машинного навчання та евристичних підходів. Актуальність дослідження зумовлена стрімким розвитком ігрової індустрії та потребою у персоналізації ігрового досвіду [1-3]. Основна увага приділяється швидкості реакції системи на зміну навичок гравця, аналізу поведінкових метрик (телеметрії смертей, часу бездіяльності, частоти вводу) та збереженню стану "поток" шляхом непомітного коригування параметрів 3D-платформера в реальному часі.

Об'єкт дослідження: процес управління рівнем складності комп'ютерної гри у реальному часі.

Предмет дослідження: системи динамічної адаптації складності (DDA) на основі евристичних методів, алгоритмів Random Forest, багатошарового перцептрону та навчання з підкріпленням.

Мета роботи: розроблення та порівняльне дослідження підсистеми динамічної адаптації складності рівнів у 3D-платформері на базі ігрового рушія Unity, яка в реальному часі аналізує поведінкові метрики гравця і непомітно коригує параметри ігрового середовища, утримуючи гравця у стані оптимальної залученості.

Методи дослідження: просторова кластеризація координат смертей, аналіз часових рядів поведінкових метрик, навчання на основі Random Forest та багатошарового перцептрону, навчання з підкріпленням із застосуванням алгоритму PPO в межах двоагентної архітектури, натхненної підходами асиметричної самогри (Asymmetric Self-Play).

Результати роботи: розроблено 3D-платформер як науковий полігон; реалізовано систему збору поведінкової телеметрії (смерті, час бездіяльності, частота вводу, тремтіння камери, час відновлення); побудовано і протестовано чотири версії DDA: базову (без адаптації), евристичну, на основі Random Forest та MLP, а також RL-систему з двома агентами (BotAgent і DirectorAgent); проведено А/В-аналіз ефективності підходів за показниками Completion Rate, середньою кількістю смертей і суб'єктивною якістю змін.

Практичне значення: створена система є відкритим, відтворюваним дослідницьким середовищем для вивчення алгоритмів DDA у 3D-іграх. Архітектура побудована на принципах SOLID і дозволяє підключати нові стратегії адаптації без змін у коді перешкод. Підхід до непомітного застосування змін через екранні переходи може бути безпосередньо використаний у комерційних проєктах.

КЛЮЧОВІ СЛОВА: ДИНАМІЧНА АДАПТАЦІЯ СКЛАДНОСТІ, МАШИННЕ НАВЧАННЯ, НАВЧАННЯ З ПІДКРІПЛЕННЯМ, ВИПАДКОВИЙ ЛІС, БАГАТОШАРОВИЙ ПЕРЦЕПТРОН, ІГРОВИЙ РУШІЙ UNITY, ПОВЕДІНКОВА ТЕЛЕМЕТРІЯ, АСИМЕТРИЧНА САМОГРА.

Перелік використаних літературних джерел:

1. Global Games Market Report [Електронний ресурс] / Newzoo. Amsterdam, 2025. 67 p. URL: https://investgame.net/wp-content/uploads/2025/09/2025_Newzoo_Free_Global_Games_Market_Report.pdf (дата звернення: 28.04.2026)
2. Zohaib M. Dynamic Difficulty Adjustment (DDA) in Computer Games: A Review / M. Zohaib // Advances in Human-Computer Interaction. 2018. Vol. 2018. P. 1-12.
3. Csikszentmihalyi M. Flow: The Psychology of Optimal Experience. New York: Harper & Row, 1990. 303 p.

ABSTRACT

Vereshchak B. O., Patereha Y. I. Development of a system for dynamic adaptation of game level difficulty. – Lviv Polytechnic National University, Lviv, 2026

Extended annotation

The work is devoted to a comparative analysis of dynamic difficulty adjustment (DDA) methods in computer games. The research focuses on analyzing the effectiveness of machine learning algorithms and heuristic approaches. The relevance of the study is driven by the rapid development of the gaming industry and the need for personalized gaming experiences [1-3]. The main attention is paid to the system's reaction speed to changes in player skills, the analysis of behavioral metrics (death telemetry, idle time, input frequency), and maintaining the "flow" state by seamlessly adjusting 3D platformer parameters in real time.

Object of research: the process of real-time difficulty management in a computer game.

Subject of research: Dynamic Difficulty Adjustment (DDA) systems based on heuristic methods, Random Forest, multi-layer perceptron, and reinforcement learning algorithms.

Purpose of the work: development and comparative study of a dynamic difficulty adjustment subsystem for a 3D platformer in Unity Engine that analyses player behavioral metrics in real time and imperceptibly modifies game environment parameters to maintain the player in a state of optimal engagement.

Research methods: spatial clustering of death coordinates, time-series analysis of behavioral metrics, Random Forest and multi-layer perceptron supervised learning, reinforcement learning with the PPO algorithm within a two-agent architecture inspired by asymmetric self-play approaches.

Results: a 3D platformer was developed as a scientific testbed; a behavioral telemetry collection system was implemented (deaths, idle time, input rate, camera jitter, time-to-recover); four DDA versions were built and tested: baseline (no adaptation), heuristic, Random Forest / MLP, and an RL system with two agents

(BotAgent and DirectorAgent); A/B analysis of approaches was performed using Completion Rate, average death count, and subjective quality of changes.

Practical significance: the created system is an open, reproducible research environment for studying DDA algorithms in 3D games. The architecture follows SOLID principles and allows new adaptation strategies to be plugged in without modifying obstacle code. The approach of applying changes invisibly during screen transitions can be directly adopted in commercial projects.

KEYWORDS: DYNAMIC DIFFICULTY ADJUSTMENT, DDA, MACHINE LEARNING, REINFORCEMENT LEARNING, RANDOM FOREST, MULTILAYER PERCEPTRON, UNITY GAME ENGINE, BEHAVIORAL TELEMETRY, ASYMMETRIC SELF-PLAY.

List of references:

1. Global Games Market Report [Electronic resource] / Newzoo. Amsterdam, 2025. 67 p. URL: https://investgame.net/wp-content/uploads/2025/09/2025_Newzoo_Free_Global_Games_Market_Report.pdf (date of application: 28.04.2026)
2. Zohaib M. Dynamic Difficulty Adjustment (DDA) in Computer Games: A Review / M. Zohaib // Advances in Human-Computer Interaction. 2018. Vol. 2018. P. 1-12.
3. Csikszentmihalyi M. Flow: The Psychology of Optimal Experience. New York: Harper & Row, 1990. 303 p.

ЗМІСТ

ВСТУП.....	12
1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД СИСТЕМИ ДИНАМІЧНОЇ АДАПТАЦІЇ СКЛАДНОСТІ.....	18
1.1 Проблематика балансування складності і теорія потоку.....	18
1.2 Економічна вартість невирішеної проблеми статичного балансування	19
1.3 Огляд існуючих систем динамічної адаптації складності.....	21
1.4 Методи машинного навчання у DDA	23
1.5 Психологічні та поведінкові метрики для оцінки стану гравця	27
1.6 Огляд технологій і інструментів	28
Висновки до розділу 1	29
2. МОДЕЛЮВАННЯ ТА ПРОЄКТУВАННЯ СИСТЕМИ ДИНАМІЧНОЇ АДАПТАЦІЇ СКЛАДНОСТІ.....	30
2.1 Системні вимоги до розроблювальної системи.....	30
2.2 Математична модель евристичного DDA	32
2.3 Математична модель ML-підходу.....	34
2.4 Математична модель RL-системи.....	36
2.5 Архітектурні моделі та UML-діаграми системи.....	38
Висновки до розділу 2	40
3. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ІНТЕГРАЦІЯ МАШИННОГО НАВЧАННЯ	42
3.1 Архітектура системи і принципи SOLID.....	42
3.2 Реалізація ігрового ядра	43
3.3 Підсистема збору телеметрії.....	44
3.4 Реалізація евристичної DDA-системи	46
3.5 ML-конвеєр та навчання моделей	48

3.6 Реалізація RL-системи.....	55
Висновки до розділу 3	56
4. ТЕСТУВАННЯ, А/В АНАЛІЗ ТА ОЦІНКА ЕФЕКТИВНОСТІ DDA СИСТЕМИ.....	58
4.1 Методика тестування та організація А/В-груп.....	58
4.2 Обмеження дослідження та загрози достовірності	59
4.3 Результати тестування евристичного DDA.....	60
4.4 Результати ML-моделей.....	61
4.5 Результати RL-тренування.....	62
4.6 Порівняльний аналіз підходів.....	64
Висновки до розділу 4	66
ВИСНОВКИ	68
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	71
ДОДАТОК А	75
ДОДАТОК Б	92

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

AD (Average Deaths) – середня кількість смертей гравця за ігрову сесію

AI (Artificial Intelligence) – штучний інтелект

Adam (Adaptive Moment Estimation) – алгоритм адаптивної оцінки моментів (метод оптимізації)

API (Application Programming Interface) – інтерфейс програмування додатків

APM (Actions Per Minute) – кількість дій за хвилину (частота вводу)

CR (Completion Rate) – частка успішно подоланих перешкод (показник проходження)

DDA (Dynamic Difficulty Adjustment) – динамічна адаптація складності

DFD (Data Flow Diagram) – діаграма потоків даних

EDA (Exploratory Data Analysis) – розвідувальний аналіз даних

FPS (Frames Per Second) – кількість кадрів на секунду

FS (Flow Score) – суб'єктивна оцінка стану потоку (залученості гравця)

GaaS (Games as a Service) – концепція «ігри як сервіс»

IS (Invisibility Score) – суб'єктивна оцінка непомітності змін складності

MAE (Mean Absolute Error) – середня абсолютна похибка

MDP (Markov Decision Process) – марковський процес прийняття рішень

ML (Machine Learning) – машинне навчання

MLP (Multi-Layer Perceptron) – багатошаровий перцептрон

MSE (Mean Squared Error) – середньоквадратична похибка

NDJSON (Newline-Delimited JSON) – формат JSON із розділенням рядками

ONNX (Open Neural Network Exchange) – відкритий формат обміну нейронними мережами

PPO (Proximal Policy Optimization) – алгоритм оптимізації наближеної політики

R² (R-squared) – коефіцієнт детермінації

ReLU (Rectified Linear Unit) – лінійна випрямляльна функція активації

RF (Random Forest) – метод випадкового лісу

RL (Reinforcement Learning) – навчання з підкріпленням

SHAP (SHapley Additive exPlanations) – метод оцінки важливості ознак (величини Шеплі)

SOLID – акронім п'яти базових принципів об'єктно-орієнтованого проєктування

ST (Session Time) – середній час ігрової сесії

UML (Unified Modeling Language) – уніфікована мова моделювання

URP (Universal Render Pipeline) – універсальний конвеєр рендерингу Unity

ВСТУП

Сучасна ігрова індустрія є одним із найбільш динамічних і технологічно містких секторів світової цифрової економіки. За даними аналітичної компанії Newzoo, світовий ринок відеоігор продовжує стабільно зростати і за прогнозами у 2027 році перевищить обсяг у 200 мільярдів доларів [1]. Проте разом із фінансовим та технологічним зростанням ринку суттєво ускладнилась і проблема утримання аудиторії (Player Retention). У зв'язку з переходом індустрії до моделі «ігри як сервіс» (Games as a Service, GaaS), фінансовий успіх продукту залежить не лише від кількості проданих копій, а й від довготривалої залученості користувачів. За різними оцінками, від 35 до 40 відсотків гравців припиняють проходження гри вже після перших годин [1, 2]. Дослідження показують, що однією з головних причин такого відтоку є відчуття гострої невідповідності між власним рівнем навичок гравця і запропонованим розробниками рівнем складності [2].

Ця проблема набуває особливої гостроти у жанрах, що вимагають високої просторової координації та швидкості реакції, зокрема у 3D-платформерах. У таких іграх ціна помилки є надзвичайно високою: невдалий стрибок, неправильний розрахунок часу або неочікувана пастка можуть призводити до циклічних смертей. Повторення одного й того самого ігрового відрізка десятки разів підряд швидко переводить гравця з комфортного стану захоплення у стан гострої фрустрації, що неминуче призводить до втрати інтересу та закриття гри [26].

Варто зазначити, що проблема відтоку гравців має не лише технічний, а й економічний вимір. За даними дослідження Kochava (2023), середня вартість залучення одного платного гравця у мобільному та PC-сегменті перевищує 3–5 доларів США, тоді як повернути гравця, який покинув гру, значно дорожче, ніж утримати його від самого початку [2]. У контексті GaaS-моделі, де основний дохід генерується через внутрішньоігрові покупки та підписки, втрата гравця впродовж перших сесій означає пряму фінансову втрату незалежно від якості

подальшого контенту. Це робить задачу автоматичного утримання аудиторії на ранніх етапах проходження одним із пріоритетних інженерних напрямків у сучасній ігровій розробці.

Концепцію оптимального стану залученості, до якого прагнуть ігрові дизайнери, фундаментально описано у теорії потоку (Flow Theory) американського психолога Міхая Чіксентміхаї [3]. Згідно з нею, стан потоку досягається тоді, коли рівень складності завдання відповідає рівню навичок виконавця. Якщо складність значно перевищує навички, виникає психологічна тривога та фрустрація; якщо ж навпаки, навички гравця переростають складність завдання, настає апатія та нудьга. Традиційний підхід із жорстко заданими рівнями складності («Легко», «Нормально», «Складно») виявився неефективним, оскільки навички гравця не є статичними: вони зростають у міру проходження гри, але водночас можуть знижуватися через втому чи втрату концентрації. Відповідно, практичне і вкрай нетривіальне завдання сучасного геймдизайну полягає в тому, щоб утримувати гравця у вузькому динамічному коридорі між нудьгою та фрустрацією впродовж усієї ігрової сесії [27].

Системи динамічної адаптації складності (Dynamic Difficulty Adjustment, DDA) з'явилися як технічна відповідь на цей виклик. Вперше цей термін набув широкого вжитку у роботах Гунікке [4], а також Чарлса та Блека [5], хоча механізми адаптації використовувались і в ранніх аркадних іграх. Мета DDA полягає у фоновому моніторингу успішності гравця та зміні параметрів ігрового світу у реальному часі без його відома. Сучасний стан галузі характеризується суперечністю між потребою у персоналізованому досвіді й обмеженістю більшості комерційних реалізацій. Класичним прикладом є система прихованого ранжування (Hidden Rank) у грі Resident Evil 4, де гра непомітно обчислює оцінку успішності гравця (від 0 до 9999) на основі його влучності та кількості отриманої шкоди, динамічно змінюючи агресивність ворогів та кількість знайдених ресурсів. Іншим відомим прикладом є штучний режисер (AI Director) у Left 4 Dead, який будує графік емоційної напруги гравців, чергуючи періоди інтенсивних атак із періодами спокою [6]. Ці правила, хоч і ефективні у межах

конкретної гри, проте мають суттєвий недолік: вони розробляються вручну (тобто є жорстко запрограмованими - Hardcoded), є надзвичайно дорогими у балансуванні, вимагають місяців тестування і, головне, не можуть переноситися на інші ігрові проекти або адаптуватися до нестандартних патернів поведінки гравців [26, 27].

Академічна сфера пропонує значно ширший і більш універсальний погляд на проблему DDA через застосування методів штучного інтелекту. Застосування алгоритмів класифікації та регресії для адаптації складності описано у роботах Зохайба [2], Кемпбела та Ку [7] та інших авторів. Зокрема, алгоритми на основі дерев рішень, а також багатошаровий перцептрон, здатні виявляти приховані закономірності в телеметрії гравців і формувати гнучкі правила адаптації на основі реальних ігрових сесій. Проте методи навчання з учителем вимагає попередньо розмічених наборів даних, що обмежує їхню автономність.

Окремим дискусійним питанням у сфері DDA залишається так звана «проблема прозорості» (transparency problem): гравці, які усвідомлюють факт маніпуляції складністю, часто сприймають це як порушення чесності гри і втрачають мотивацію. Зокрема, дослідження Нака та Дрехена [10] показують, що суб'єктивне сприйняття адаптації значною мірою залежить від її «непомітності» – тобто від того, наскільки плавно і несподівано вона відбувається з точки зору гравця. Цей поведінковий аспект суттєво ускладнює проектування DDA-систем: недостатньо лише точно передбачити поточний стан гравця – не менш важливо обрати момент і спосіб застосування змін так, щоб вони залишалися непоміченими. Саме ця вимога визначила архітектурне рішення даної роботи щодо буферизації змін і їхнього застосування виключно під час природних візуальних переходів між ігровими відрізками.

З огляду на це, перспективним напрямком сучасної ігрової інженерії є навчання з підкріпленням (RL). У цій парадигмі спеціальний програмний агент – «Режисер» (Director) навчається безпосередньо максимізувати показник залученості гравця шляхом спроб і помилок, формулюючи DDA як складну математичну задачу марковського процесу прийняття рішень [8, 9]. Однак

більшість таких досліджень проводяться на 2D-задачах із простим простором станів (Pac-Man, Flappy Bird, бійцівські ігри), де спостережуваний стан гри легко описати невеликим вектором ознак [26]. Для 3D-платформерів ситуація принципово інша: простір можливих станів і координат смертей є тривимірним, фізика руху – неперервною, а поведінка гравця визначається глибоким комплексом не лише технічних, а й психологічних чинників. Порівняльних досліджень, які б охоплювали перенесення RL-агентів у комплексні 3D-середовища, на сьогодні практично немає.

Таким чином, наявний стан галузі характеризується двома взаємопов'язаними прогалинами. По-перше, більшість академічних досліджень DDA на основі машинного навчання обмежені 2D-задачами та не враховують тривимірної просторової структури й психологічних індикаторів стану гравця. По-друге, відсутнє відкрите відтворюване дослідницьке середовище, яке дозволяло б порівнювати різні алгоритмічні підходи до DDA в ідентичних умовах на базі реального 3D-платформера. Усунення цих прогалин і визначає актуальність даної роботи.

Для вирішення сформульованих задач обрано ігровий рушій Unity (версія 6.3 LTS) як платформу розробки, оскільки він поєднує широкі можливості для реалізації 3D-сцен, наявність офіційного інструментарію для навчання агентів (ML-Agents Toolkit) та підтримку розгортання ONNX-моделей безпосередньо у середовищі виконання через пакет Unity Sentis [19, 20, 21]. Робота виконана з дотриманням принципів відтворюваності дослідження: увесь код, навчальні дані та конфігурації агентів розміщені у публічному репозиторії, що дозволяє незалежно відтворити або розширити отримані результати.

Об'єктом дослідження є процес управління рівнем складності комп'ютерної гри у реальному часі.

Предметом дослідження є системи динамічної адаптації складності, побудовані на основі евристичних алгоритмів, методу випадкового лісу (Random Forest), багат шарового перцептрону (MLP) та навчання з підкріпленням (RL).

Метою роботи є розроблення та порівняльне дослідження підсистеми DDA для 3D-платформера в Unity, яка в реальному часі аналізує поведінкові метрики гравця і непомітно коригує ігрове середовище для підтримки стану оптимальної залученості.

Для досягнення мети сформульовано такі задачі:

- 1) провести аналіз існуючих DDA-систем і методів машинного навчання, застосовуваних у ігровому контексті;
- 2) розробити підсистему збору поведінкової телеметрії та визначити вектор ознак для ML-моделей;
- 3) реалізувати та налагодити евристичну DDA-систему з просторовою кластеризацією смертей;
- 4) побудувати ML-конвеєр для навчання Random Forest і MLP-моделей на зібраних даних та інтегрувати їх в Unity;
- 5) спроектувати та реалізувати RL-систему з агентами BotAgent і DirectorAgent на основі підходів, натхнених асиметричною самогрою (Asymmetric Self-Play);
- 6) провести А/В-тестування чотирьох варіантів DDA і зробити висновки щодо їхньої порівняльної ефективності.

Методи дослідження: просторова кластеризація (K-Means-подібний центроїдний алгоритм), ансамблеве навчання (Random Forest), навчання нейронної мережі зі зворотним поширенням помилки (MLP), навчання з підкріпленням (PPO), підходи на основі асиметричної самогри.

Наукова новизна одержаних результатів полягає у створенні відтворюваного експериментального середовища для порівняльного дослідження DDA-підходів у 3D-платформерах. На відміну від більшості існуючих академічних робіт, орієнтованих на 2D-задачі зі спрощеним простором станів, у даній роботі вирішено три взаємопов'язані задачі. По-перше, набір спостережуваних метрик розширено за рахунок просторової кластеризації з урахуванням останньої безпечної позиції (Last Safe Position), часу когнітивного відновлення після смерті (Time to Recover), частоти вводу (APM) та тремтіння

камери (Jitter) – показників, що відображають психологічний стан гравця і відсутніх у порівнюваних реалізаціях [27, 28]. По-друге, реалізовано наскрізний ML-конвеєр: від збору даних за принципом «людина-в-контурі» (Human-in-the-Loop) до розгортання навченої моделі безпосередньо в середовищі виконання Unity без сторонніх Python-залежностей. По-третє, реалізовано RL-архітектуру з двома агентами, натхненну принципом асиметричної самогри, що дозволяє проводити сотні тисяч кроків тренування у прискореному режимі без участі реального гравця [28, 30].

Практичне значення роботи полягає у тому, що розроблений прототип є повністю відтворюваним і може слугувати відкритим дослідницьким середовищем (полігоном) для подальшого вивчення алгоритмів DDA науковою спільнотою. Архітектурні рішення, зокрема патерн Стратегія для DDA і механізм непомітного застосування змін через екранні переходи, орієнтовані на безпосереднє перенесення у комерційні проєкти ігрових студій. З метою забезпечення відтворюваності дослідження вихідний код проєкту, навчальні набори даних, конфігураційні файли агентів та інструкції з відтворення експерименту розміщено у публічному репозиторії [37]. Демонстраційна збірка гри з описом керування та поясненням роботи адаптивної системи доступний на платформі itch.io [38].

1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД СИСТЕМИ ДИНАМІЧНОЇ АДАПТАЦІЇ СКЛАДНОСТІ

1.1 Проблематика балансування складності і теорія потоку

Балансування складності є одним із ключових завдань геймдизайну і водночас одним із найменш формалізованих. У традиційній практиці дизайнер встановлює кілька фіксованих рівнів складності («легко», «нормально», «важко»), які гравець обирає перед початком сесії. Такий підхід ігнорує два важливих факти: по-перше, можливості одного й того самого гравця змінюються з часом і залежать від контексту; по-друге, однакова механіка сприймається різними гравцями як діаметрально протилежна за складністю.

Теоретичну основу для розуміння оптимального ігрового досвіду закладає концепція потоку Чіксентміхаї [3, 27]. Психолог спостерігав, що стан максимальної концентрації і задоволення від діяльності виникає у вузькій зоні, де рівень виклику C та рівень навичок S є порівнянними (рис. 1.1).

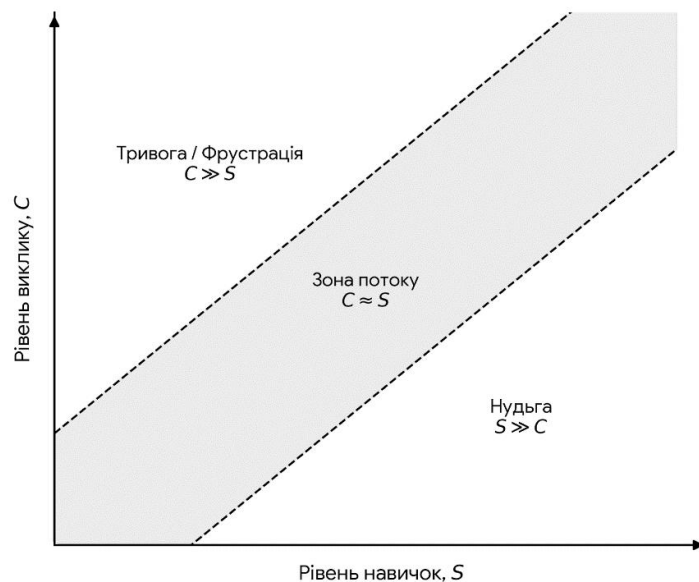


Рис. 1.1 Класичний графік теорії потоку

Якщо виклик значно перевищує навички ($C \gg S$), виникає тривога або фрустрація; якщо навпаки ($S \gg C$), людина відчуває нудьгу. Формально умову знаходження в зоні потоку можна описати як:

$$|C - S| < \epsilon \quad (1.1)$$

де ϵ є допустимим відхиленням, яке залежить від індивідуальних особливостей гравця. Прикладна задача DDA полягає у тому, щоб динамічно наближати C до поточного S , не розкриваючи механізм адаптації гравцеві, адже усвідомлення маніпуляції знищує ігровий досвід [4].

Зв'язок між смертями і рівнем фрустрації нелінійний. Перша серія смертей підряд часто сприймається як навчання, проте після певного порогу починається накопичення когнітивної втоми. Ключовим показником є не абсолютна кількість смертей, а їхня просторова локалізація: якщо гравець помирає в одному й тому самому місці, це вказує на конкретну технічну перешкоду, а не на загальну невідповідність рівня складності. Саме це спостереження лежить в основі кластерного підходу до DDA, реалізованого у цій роботі.

Додатковими психологічними індикаторами є поведінкові сигнали, що передують явній фрустрації. Зростання тремтіння камери (Camera Jitter) і різке підвищення частоти вводу (APM) свідчать про тривогу і паніку. Тривалі паузи після смерті (idle time, Time to Recover) вказують на когнітивне перевантаження або демотивацію. Аналіз цих сигналів дозволяє DDA-системі втрутитись ще до того, як гравець вийде з гри [10, 27]. Утім, психологічна природа цих сигналів набуває практичної ваги лише тоді, коли їхнє ігнорування спричиняє задокументовані наслідки для конкретних ігрових проєктів.

1.2 Економічна вартість невирішеної проблеми статичного балансування

Проблема втрати аудиторії через невідповідний рівень складності має не лише теоретичний характер, а й призводить до прямих вимірюваних економічних наслідків для індустрії відеоігор. За даними галузевого звіту GameAnalytics за 2026 рік, який охоплює дані 16 262 відеоігор, медіанний показник утримання гравців першого дня становить близько 22%. Це означає, що понад три чверті нових користувачів відмовляються від гри протягом першої

добу після її встановлення. На 30-й день цей показник для 75% ігор падає нижче 1,8% [31]. Хоча втрата зацікавленості зумовлена комбінацією факторів, дослідження М. Зохайба та систематичний огляд Л. Вітекової зі співавторами [2, 26] ідентифікують невідповідність між складністю і навичками гравця як один із основних чинників ранньої відмови від гри.

На підтвердження цієї тези доцільно розглянути кілька задокументованих прикладів із комерційної ігрової практики:

Mighty No. 9 (2016). Концептуальний наступник класичного Mega Man зібрав під час кампанії з громадського фінансування понад 3,84 мільйона доларів США від більш ніж 67 тисяч добровольців. Після виходу гра отримала оцінку Metacritic на рівні у діапазоні 48–55 балів зі 100 залежно від платформи. Рецензенти від видань GameSpot, Destructoid та IGN окремо вказали на відсутність прогресивної кривої складності як на ключовий недолік: гра не навчала поступово, а ставила миттєві смерті (insta-kills) і непрогнозовані пастки [32]. Характерно, що навіть на найнижчому рівні складності механіки залишалися бар'єром для широкої аудиторії.

Crash Bandicoot, рівень «Stormy Ascent» (1996/2017). Цей рівень є прикладом від виробника серії, алгоритми адаптації якої будуть детальніше розглянуті в наступному підрозділі. Тейлор Курасакі (Taylor Kurosaki), дизайнер рівня, у 2017 році публічно визнав: «Тоді я ще не добре розумів, що таке крива складності, і рівень вийшов нестерпно важким навіть для мене» [33]. Рівень був виключений із гри безпосередньо перед релізом 1996 року, лишившись доступним лише через спеціальні коди модифікації GameShark. Це свідчить про глибоке усвідомлення студією Naughty Dog руйнівного ефекту дисбалансу складності. Рівень офіційно повернули до гри лише у 2017 році.

Battletoads, рівень «Turbo Tunnel» (1991). Класичний приклад з академічної літератури з дизайну ігрового досвіду [34]. Третій рівень гри вимагає від гравця запам'ятовування послідовностей перешкод майже без часу на реакцію, що прямо суперечить умові $C \approx S$. Дослідники вказують на руйнівний вплив дисбалансу на комерційну долю серії, яка була по суті мертвою понад 30 років.

Sonic the Hedgehog 2006 (SEGA). Ювілейний реліз із бюджетом понад 20 мільйонів доларів отримав оцінку Metacritic у 43–46 балів [35]. Рецензенти визначили «проблемні ракурси камери, що призводять до несправедливих ігрових поразок» та «раптові стрибки складності без підготовки гравця» як систематичні помилки дизайну. Проблема поглиблювалася технічними недоліками, що демонструє комплексний характер втрати аудиторії: технічна нестабільність і дисбаланс складності посилюють одне одного.

Протилежний приклад: Cuphead (Studio MDHR, 2017). Для повноти аналізу необхідно розглянути гру, яка успішно продавалася з умисно високим і незмінним рівнем складності без систем DDA. Продажі Cuphead до 2022 року перевищили 6 мільйонів копій [36]. Ключовою відмінністю від провальних прикладів є те, що складність гри є послідовною, чесною і наочно відображається в архітектурі рівнів: гравці розуміють правила і можуть аналізувати свої помилки. C є стабільним, а S гравця зростає через повторення. Таким чином, Cuphead не спростовує необхідність DDA, а уточнює умову: фіксована складність є прийнятною лише тоді, коли крива складності розроблена з надзвичайною точністю для вузькоспеціалізованої цільової аудиторії.

Наведені приклади дозволяють сформулювати дослідницьке питання цієї роботи з практичного погляду: чи здатна автоматична адаптація параметрів ігрового середовища на основі поведінкових метрик гравця підтримувати стан потоку для різномірної аудиторії, не жертвуючи при цьому відчуттям майстерності та без видимих маніпуляцій? Відповідь на це питання потребує порівняльного дослідження декількох підходів до DDA.

1.3 Огляд існуючих систем динамічної адаптації складності

Системи DDA присутні в іграх із 1980-х років, проте лише з 2000-х набули чіткого концептуального оформлення. Огляд репрезентативних реалізацій дає змогу виявити системні обмеження існуючих підходів і обґрунтувати напрямки вдосконалення.

1.3.1 Евристичні системи на основі лічильників

Resident Evil 4 (Capcom, 2005) використовує систему прихованого ранжування (Hidden Rank), яку пізніше дослідники охрестили «сумкою купця» [6]. Алгоритм відстежує поточний бойовий рейтинг на основі кількості вбитих ворогів і отриманих ударів. Залежно від рейтингу змінюється щільність ворогів у наступному сегменті. Система повністю детермінована: кожне значення рейтингу відповідає конкретному набору параметрів. Перевага цього підходу полягає у передбачуваності й легкості налаштування, недолік, суттєвий для задач даного дослідження, полягає у тому, що система не враховує просторовий контекст і не реагує на психологічний стан гравця.

Left 4 Dead (Valve, 2008) реалізує штучний режисер (AI Director), що керує хвилями супротивників у реальному часі на основі кількох параметрів: здоров'я команди, кількості вбитих ворогів і рівня стресу. Режисер може посилювати або послаблювати натиск, враховуючи колективний стан групи [6]. Важливою особливістю є використання «спокійних фаз», які навмисно дозволяють гравцям відновитись перед наступною хвилею. Проте і цей алгоритм залишається суто евристичним і не здатен навчатись на нових паттернах поведінки.

Серія Crash Bandicoot (Activision) використовує простіший підхід: якщо гравець помирає на одному рівні кілька разів поспіль, гра пропонує контрольну точку або маску невразливості. Це приклад найпростішої форми DDA, заснованої виключно на лічильнику смертей без жодного просторового аналізу. Порівняльну характеристику розглянутих систем наведено в таблиці 1.1.

Таблиця 1.1

Порівняльна характеристика існуючих DDA-систем

Система	Тип алгоритму	Метрики	Обмеження
Resident Evil 4	Евристика (лічильник)	Бойовий рейтинг	Без просторового аналізу, детерміновано
Left 4 Dead	Евристика (правила)	Здоров'я, стрес	Кооперативний жанр, не переноситься

Продовження Таблиця 1.1

Crash Bandicoot	Лічильник смертей	Смерті підряд	Без будь-якого ML, лише поріг
Ця робота	Гібридний (Heuristic + RF + MLP + RL)	8 поведінкових метрик	Навчання на реальних даних, 3D

Як видно з таблиці 1.1, усі розглянуті евристичні системи мають спільну структурну ваду: правила адаптації прописані розробником заздалегідь і не оновлюються в процесі роботи. Така система не розрізняє гравців з різним стилем: вона однаково реагує на десяту смерть поспіль досвідченого гравця, що просто не уважний, і недосвідченого, який відчуває справжню фрустрацію. Усунення цього обмеження є основною мотивацією для застосування методів машинного навчання.

1.4 Методи машинного навчання у DDA

Застосування машинного навчання до DDA відкриває якісно нові можливості, оскільки дозволяє системі формувати правила адаптації автоматично на основі накопичених даних, а не вручну запрограмованих умов [26]. Виділяють три основні підходи: навчання з учителем, навчання без учителя і навчання з підкріпленням. Загальну архітектуру інтеграції ML-моделей у логіку ігрового рушія наведено на аркуші 5 графічного матеріалу.

Для розв'язання задачі DDA у роботі обрано три методи, що представляють якісно різні парадигми машинного навчання, а не є варіантами в межах однієї. Такий підхід дозволяє провести порівняльне дослідження не лише за точністю прогнозу, а й за принципово різними механізмами навчання та вимогами до вхідних даних.

Перша парадигма представлена алгоритмом Random Forest як класичним методом ансамблевого навчання з учителем. Серед інших методів з учителем, придатних для задачі регресії, зокрема SVM, k-NN і Logistic Regression, Random Forest обрано з кількох причин: він не вимагає попередньої нормалізації вхідних ознак і демонструє стійку роботу на відносно малих наборах даних, що є

критичною умовою для телеметрії, зібраної обмеженою групою тестувальників. Практично важливою є також можливість трансляції навченої sklearn-моделі у чистий C#-код через інструмент m2cgen [18], що виключає залежності від мови Python у середовищі виконання Unity. Методи Gradient Boosting і XGBoost, що зазвичай перевершують Random Forest на стандартних табличних еталонних наборах даних, у даному контексті поступаються через суттєво вищу обчислювальну вартість навчання на малих вибірках із шумом та ускладнену інтерпретацію важливості ознак, що є важливим для аналізу метрик гравця.

Друга парадигма представлена багатошаровим перцептроном (MLP) як нейромережовим підходом. Мета його включення є принципово іншою: не пошук найкращого методу навчання з учителем, а перевірка гіпотези про те, чи існують нелінійні залежності між поведінковими метриками гравця, які ансамблеві дерева апроксимують гірше за нейронні мережі. Навчання проводиться на тих самих даних, що й Random Forest, що забезпечує коректне пряме порівняння.

Третя парадигма, навчання з підкріпленням на основі алгоритму PPO [17], є принципово відмінною від двох попередніх: вона не потребує заздалегідь розмічених прикладів від людини. Натомість цільова функція, стан потоку гравця, кодується безпосередньо у функцію нагороди агента. Це усуває головне обмеження підходів навчання з учителем при масштабуванні: зібрати достатній обсяг явно розмічених прикладів для повного покриття простору ігрових ситуацій на практиці неможливо. Вибір саме цієї трійки методів утворює мінімальний показовий набір для порівняльного дослідження, а розширення його за рахунок інших алгоритмів (наприклад, CatBoost або архітектур трансформерів) є природним напрямом подальших досліджень.

1.4.1 Random Forest для регресії множника складності

Random Forest (Breiman, 2001) [11] є ансамблевим методом, що поєднує передбачення множини дерев рішень для отримання приблизної оцінки. Кожне дерево навчається на підвибірці даних і підмножині ознак, що знижує дисперсію порівняно з одиночним деревом рішень. Для задачі DDA проблему було

сформульовано як регресію: за вектором x поведінкових ознак гравця модель передбачає оптимальний множник складності $\hat{y}$, де $\hat{y} = 1$ відповідає поточній складності, $\hat{y} > 1$ означає полегшення, а $\hat{y} < 1$ ускладнення.

Щоб оцінити, наскільки доцільною є відмова від нейронних мереж на користь ансамблевих методів, поряд із Random Forest розглядається другий підхід з учителем, але з іншою архітектурою, який навчений на тих самих даних і вирішує ту саму регресійну задачу.

1.4.2 Багат шаровий перцептрон (MLP)

Багат шаровий перцептрон [13] є найпростішою архітектурою глибокого навчання (рис. 1.2).

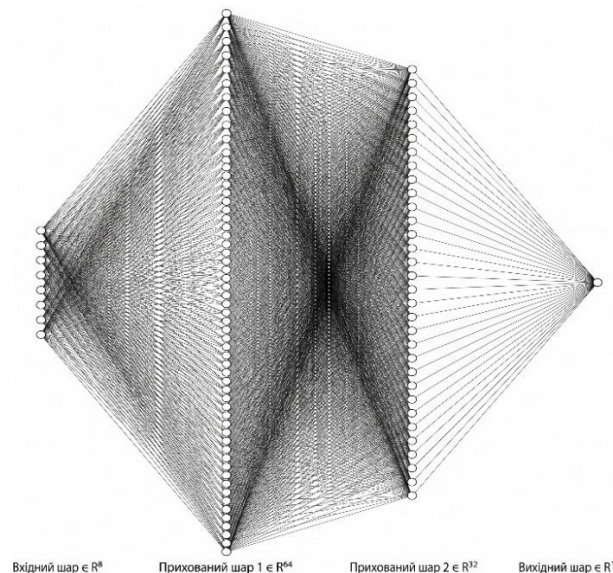


Рис. 1.2 Схеми архітектури нейромережі

У контексті роботи застосовується MLP із двома прихованими шарами (64 і 32 нейрони відповідно) і функцією активації ReLU. Вихідний шар містить один нейрон із лінійною активацією і передбачає множник складності. MLP навчається на тих самих даних, що й Random Forest, методом мінімізації середньоквадратичної помилки (MSE) за допомогою оптимізатора Adam.

На відміну від Random Forest, MLP потребує стандартизації вхідних даних і більшого обсягу навчальних вибірок для досягнення порівнянної якості. Проте теоретично MLP здатний гнучкіше апроксимувати нелінійні залежності у просторі ознак і природно розширюється до більш складних архітектур. Навчена

модель конвертується у формат ONNX і виконується у Unity через пакет Unity Sentis без будь-яких зовнішніх залежностей.

Обидва розглянуті методи, Random Forest і MLP, є алгоритмами навчання з учителем і поділяють принципове обмеження: для їх тренування необхідний датасет із явно розміченими прикладами, де людина заздалегідь визначає бажаний множник складності для кожної ситуації. Оскільки зібрати достатній обсяг таких еталонних даних для повного покриття простору ігрових станів на практиці неможливо, виникає потреба в альтернативних парадигмах.

1.4.3 Двоагентні RL-архітектури на основі підходів асиметричної самогри (Asymmetric Self-Play)

Навчання з підкріпленням (RL) розв'язує згадану проблему розмітки, розглядаючи DDA як задачу марковського процесу прийняття рішень (MDP) [8]. Агент-«Режисер» у кожний момент часу спостерігає стан середовища s , виконує дію a (зміна параметрів перешкод і атмосфери) і отримує числову нагороду r , яку намагається максимізувати протягом усього епізоду. Ключова перевага RL полягає у тому, що цільова функція (стан потоку гравця) може бути закодована безпосередньо у функції нагороди, принципово усуваючи необхідність ручної класифікації даних людиною [29].

Оскільки проведення мільйонів навчальних ігрових сесій з реальними гравцями є неможливим, у роботі застосовано двоагентну RL-архітектуру, натхненну підходами асиметричної самогри (Asymmetric Self-Play) [9, 14].

Система складається з двох агентів: BotAgent (штучний гравець) і DirectorAgent (управляє складністю). BotAgent спочатку тренується самостійно, набуваючи навичок проходження рівня. Після цього його ваги заморожуються, і починається тренування DirectorAgent, який використовує бота як симулятор реакцій реального гравця. Схему взаємодії агентів за цим принципом у даному проєкті наведено на аркуші 6 графічного матеріалу.

Незалежно від того, яка з трьох розглянутих парадигм застосовується, якість адаптації безпосередньо визначається якістю вхідних спостережень, що

передаються моделі. Якщо спостереження не охоплюють реальних сигналів фрустрації, жоден алгоритм не зможе відреагувати своєчасно.

1.5 Психологічні та поведінкові метрики для оцінки стану гравця

Ефективність DDA залежить від якості метрик, на основі яких приймаються рішення. Якщо метрики не відображають реальний психологічний стан гравця, адаптація буде несвоєчасною або хибною. У цій роботі використовується розширений набір із восьми метрик, більшість із яких є похідними від первинних подій гри [28].

Рівень (level) визначає загальний контекст, оскільки один і той самий показник смертей на першому рівні означає абсолютно інше, ніж на третьому. Смерті з останньої точки збереження (checkpoint_deaths) є основним сигналом локальної труднощі. Смерті у кластері (cluster_deaths) відображають концентрацію проблеми у просторі. Частота вводу (input_rate, APM) вимірює рівень моторного навантаження і дає змогу розрізнити активного гравця від того, хто знаходиться у зоні комфорту. Час проходження рівня (time_spent) дає інформацію про загальний темп. Час когнітивного відновлення (time_to_recover) вимірює затримку між появою після смерті і першим рухом гравця, відображаючи глибину емоційного пригнічення. Час бездіяльності (idle_time) вказує на розгубленість або перерву у грі. Тремтіння камери (jitter) корелює з панічними рухами миші [27, 28].

Зважена сума цих метрик формує показник когнітивного навантаження (CognitiveLoad):

$$L_{\text{cog}} = 0.35 * r_d + 0.20 * r_{\text{idle}} + 0.20 * (1 - r_{\text{input}}) + 0.25 * r_{\text{atm}} \quad (1.2)$$

де r_d , r_{idle} , r_{input} є нормалізованими до інтервалу $[0, 1]$ значеннями смертей, бездіяльності і частоти вводу, а r_{atm} відображає рівень атмосферного напруження, що задається DirectorAgent, яке буде формалізовано далі. Цей показник

подається у якості однієї з ознак у вектор спостережень BotAgent і дозволяє DirectorAgent впливати на поведінку бота через симуляцію емоційного стану.

Обраний набір з восьми ознак є компромісом між охопленням релевантних сигналів і обчислювальною ефективністю в реальному часі. Більший набір ризикує перенавчанням на малих вибірках (за принципом "прокляття розмірності"), а менший – втратою інформативності щодо психологічного стану.

Запропонований набір ознак спільно з архітектурою компонентів машинного навчання формують конкретні технічні вимоги до реалізації: метрики обчислюються у реальному часі без блокування ігрового циклу, модель виконується безпосередньо всередині рушія, а дані зберігаються у форматі, придатному для подальшого навчання поза середовищем виконання. Саме ці вимоги визначили вибір технологічного стеку.

1.6 Огляд технологій і інструментів

Для реалізації проекту обрано такий технологічний стек.

Ігровий рушій Unity версії 6.3 LTS є стандартом індустрії для початкової розробки і надає повноцінну підтримку 3D-фізики, пост-обробки (Universal Render Pipeline), систему подій і властиве API для написання ігрової логіки на C#. Специфічно для проекту використовуються такі пакети: Input System (гнучке керування введенням), Cinemachine (virtual camera), ProBuilder (створення прототипів рівнів), Unity ML-Agents [16] (тренування RL-агентів), Unity Sentis (використання ONNX-моделей у середовищі виконання).

Бібліотека scikit-learn [15] для Python є основним інструментом ML-конвеєра. Вона надає реалізації Random Forest, інструменти для розбиття даних, нормалізації та оцінки метрик якості. Для навчання MLP використовується PyTorch або scikit-learn MLPRegressor з подальшим конвертуванням у ONNX через бібліотеку skl2onnx або torch.onnx.export.

Бібліотека m2cgen [18] забезпечує трансляцію навченої sklearn-моделі у класичний C#-код. Це критично важливо для ефективності в середовищі виконання: замість завантаження файлу моделі й виклику стороннього

інтерпретатора, Unity виконує чистий код без будь-яких накладних витрат на серіалізацію даних. Вихідний клас `RandomForestModel.Score(double[] args)` повертає результат за мікросекунди навіть на мобільних процесорах, проте вимагає довгої ініціалізації.

Unity ML-Agents [16] є відкритим інструментарієм для навчання RL-агентів безпосередньо у рушії. Він реалізує стандартне середовище gym, підтримує алгоритми PPO і SAC, а також дозволяє паралельно запускати кілька екземплярів середовища для прискорення збору досвіду. Навчені нейронні мережі зберігаються у форматі ONNX і виконуються за допомогою рушія тензорних обчислень Unity Sentis.

Висновки до розділу 1

У першому розділі проведено системний аналіз проблематики балансування складності у комп'ютерних іграх. Встановлено, що стан потоку є психологічно обґрунтованою цільовою функцією для DDA-системи, а більшість сучасних комерційних реалізацій обмежуються примітивними евристиками, що не враховують просторовий контекст і психологічний стан гравця.

Проведено огляд методів машинного навчання, відповідні для задачі: Random Forest, MLP і навчання з підкріпленням. Кожен метод має специфічні переваги: RF є прозорим і придатним для малих вибірок даних, MLP гнучко апроксимує нелінійності, а RL дозволяє навчатись без явної класифікації, безпосередньо оптимізуючи цільову функцію залученості.

Обґрунтовано базовий набір поведінкових метрик, що комплексно відображає стан гравця через просторові (локалізація точок загибелі) та психологічні (час когнітивного відновлення, час бездіяльності, моторна частота вводу, тремтіння камери) індикатори.

Проведено огляд технологічного стеку проєкту: Unity, C#, scikit-learn, ML-Agents, Sentis, m2cgen. Визначено, що обраний стек повністю покриває потреби всіх запланованих етапів реалізації.

2. МОДЕЛЮВАННЯ ТА ПРОЄКТУВАННЯ СИСТЕМИ ДИНАМІЧНОЇ АДАПТАЦІЇ СКЛАДНОСТІ

2.1 Системні вимоги до розроблювальної системи

На початковому етапі проєктування сформульовано дві групи вимог: технологічні (нефункціональні) і функціональні. Перша група визначає якісні характеристики системи, друга описує конкретну поведінку її компонентів. Зведені вимоги наведено в таблиці 2.1.

2.1.1 Технологічні вимоги

Продуктивність. Обчислення евристичних формул і виклик ML-моделей не повинні призводити до зниження кадрової частоти нижче 60 кадрів на секунду (FPS). Це досягається асинхронним записом телеметрії у фоновому потоці або через співпрограми, а також кешуванням вхідних масивів для нейронної мережі, що виключає динамічне виділення (алокацію) пам'яті під час виконання.

Розширюваність і підтримуваність. Архітектура має дозволяти додавання нових стратегій DDA (нових алгоритмів ML або евристик) без зміни коду перешкод. Це вимагає дотримання принципу відкритості/закритості (OCP) через архітектурний патерн “Стратегія” (Strategy).

Переносимість. Система має компілюватись під операційні системи Windows, Linux та середовище WebGL без платформи-специфічних залежностей. Модель ONNX за допомогою пакета Unity Sentis виконується на центральному процесорі (CPU) у крос-платформному режимі.

Невидимість адаптації. Зміни параметрів перешкод не повинні бути помітними гравцеві. Це досягається буферизацією змін і застосуванням їх виключно під час візуального переходу затемнення екрану (Screen Darkening), окрім випадків візуальної зміни перешкод, а саме збільшення, зменшення та зміни їхньої траєкторії руху а також при домінуванні гравця, коли миттєве підвищення складності є необхідним для підтримки стану "поток" (flow state).

2.1.2 Функціональні вимоги

Система збору телеметрії повинна фіксувати всі ключові ігрові події в реальному часі: старт і завершення рівня, смерть гравця із зазначеними просторовими координатами і причиною, активацію контрольних точок, взаємодію з перешкодами. Частота обчислення APM (дій за хвилину) і Jitter (тремтіння камери) повинна задаватись параметром змінного вікна (за замовчуванням 10 секунд).

Підсистема управління (DDA-менеджер) повинен підтримувати три режими роботи, які задаються через DDAMode: DataCollection_Baseline (збір даних без адаптації), Heuristic_Math (математичні формули) і MachineLearning_AI (виведення на основі ML-моделей). Переключення між режимами здійснюється в інспекторі Unity без необхідності перекомпіляції, або прямо у збірці проєкту, через меню налаштувань в графічному інтерфейсі користувача (GUI).

Всі динамічні перешкоди реалізують інтерфейс IDynamicObstacle і реєструються у глобальному реєстрі DDAManager.dynamicObstacles при активації. При деактивації або знищенні об'єкта реєстрація автоматично видаляється для запобігання помилок доступу.

Таблиця 2.1

Зведена таблиця системних вимог

Вимога	Значення / опис
Мінімальний FPS	60 кадрів/секунду (цільове середовище)
Затримка DDA-відповіді	Не більше наступного затемнення екрану (~0.5 с)
Обсяг журналу подій-файлу/сесія	Не більше 5 МБ (NDJSON)
Кількість типів перешкод	Не менше 4 (падаючі, рухомі, шипові, прості)
Перерахування DDAMode	3 режими: Baseline, Heuristic, ML
Кількість ознак ML	8 (level, checkpoint_deaths, cluster_deaths, input_rate, time_spent, time_to_recover, idle_time, jitter)

2.2 Математична модель евристичного DDA

Евристичний алгоритм DDA базується на просторовій кластеризації координат смертей і обчисленні зваженого впливу на найближчі перешкоди (рис. 2.1).

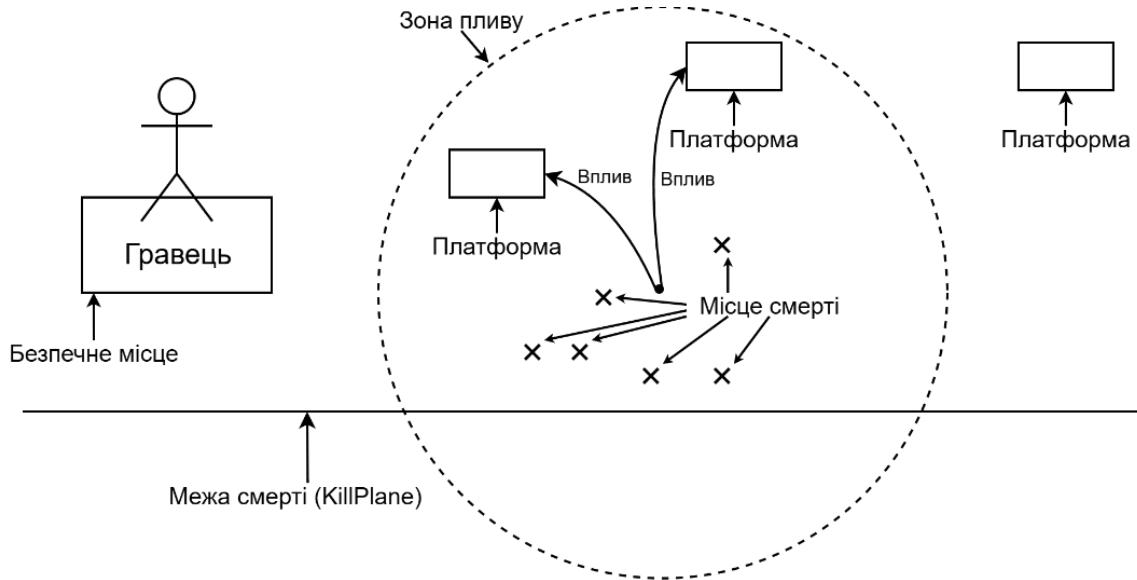


Рис. 2.1 Візуалізація просторової кластеризації впливу

Математично алгоритм складається з трьох послідовних кроків.

Крок 1. Формування кластера. Нехай $P = \{p_1, p_2, \dots, p_n\}$ є множиною позицій смертей у часовому вікні T_w . Для кожної пари перевіряється відстань за стандартною формулою евклідової відстані:

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2} \quad (2.1)$$

Якщо існує щонайменше d_{min} смертей у сфері радіуса r_{clust} з центром у останній смерті p_n , кластер вважається виявленим. У програмній реалізації використовуються параметри

- $r_{clust} = 10 \text{ м}$,
- $d_{min} = 2 \text{ смерті}$,
- $T_w = 30 \text{ с}$.

Крок 2. Обчислення центроїда. Координати центру C проблемної зони розраховуються за формулою центроїда:

$$C = \frac{1}{k} * \sum_{i=1}^k p_i \quad (2.2)$$

де k – кількість смертей, що потрапили в кластер.

Крок 3. Обчислення зваженого множника. Для кожної перешкоди O_j із позицією o_j розраховується нормалізована відстань до центра кластера:

$$d_j = dist(o_j, C) \quad (2.3)$$

Для визначення міри впливу на об'єкт залежно від його віддаленості запропоновано вагову функцію вигляду, що спадає зі збільшенням відстані:

$$w_j = \max\left(0, 1 - \frac{d_j}{r_{adapt}}\right) \quad (2.4)$$

де r_{adapt} є радіусом адаптації (за замовчуванням 10 м). Для перешкод за межами r_{adapt} вплив дорівнює нулю. Для інтеграції цих показників запропоновано формулу обчислення кінцевого множника полегшення вигляду:

$$M_{ease}(j) = lerp(1, M_{target}, w_j * weight_j) \quad (2.5)$$

де $weight_j$ є власною вагою перешкоди з діапазону $[0, 2]$, що дає змогу індивідуально налаштовувати чутливість кожної перешкоди до системи DDA. M_{target} є бажаним множителем полегшення (наприклад, 1.5 означає збільшення розміру платформи на 50 відсотків).

Ускладнення активується при домінуванні гравця (нуль смертей між двома точками збереження). У цьому випадку застосовується аналогічна формула з радіусом $r_{compl} = 20$ м і $M_{target} < 1$.

Захист від відкату. Кожна перешкода підтримує змінну `ddaProgressTracker`, що зберігає максимальний досягнутий множник. Нові множники, що збільшують труднощі, але не перевищують `ddaProgressTracker`, ігноруються до тих пір, поки глобальна складність не знизиться нижче 1.0. Це забезпечує стійку динаміку зміни складності навіть при чергуванні полегшення і ускладнення ігрового процесу.

2.3 Математична модель ML-підходу

Обидві ML-моделі (Random Forest і MLP) вирішують спільну задачу: регресію оптимального множника складності $\hat{y}$ на основі вектора ознак x . Вектор ознак має розмірність 8 і формується за рівнянням:

$$x = [level, ceckpoint_d, cluster_d, input_{rate}, time_{spent}, t_{recover}, idle_t, jitter] \quad (2.6)$$

Цільова змінна $y = desired_multiplier$ є явно розміченими даними, отриманими від тестувальників через підсистему ручного зворотного зв'язку. Коли тестувальник вважає етап гри надто складним, він натискає гарячу клавішу і встановлює бажаний множник (наприклад, 1.3). Ця дія фіксується як рядок у журналі подій (логі) з тегом *player_feedback* і стає навчальним прикладом із міткою $y = 1.3$.

Метод випадкового лісу (Random Forest) будується із T дерев. Кожне дерево h_t навчається на випадковій підвибірці даних [11]:

$$F(x) = \frac{1}{T} * \sum_{t=1}^T h_t(x) \quad (2.7)$$

Якість навчання моделі вимірюється середньоквадратичною похибкою (MSE) і коефіцієнтом детермінації R^2 , графік кривої навчання наведено на рис. 2.2.

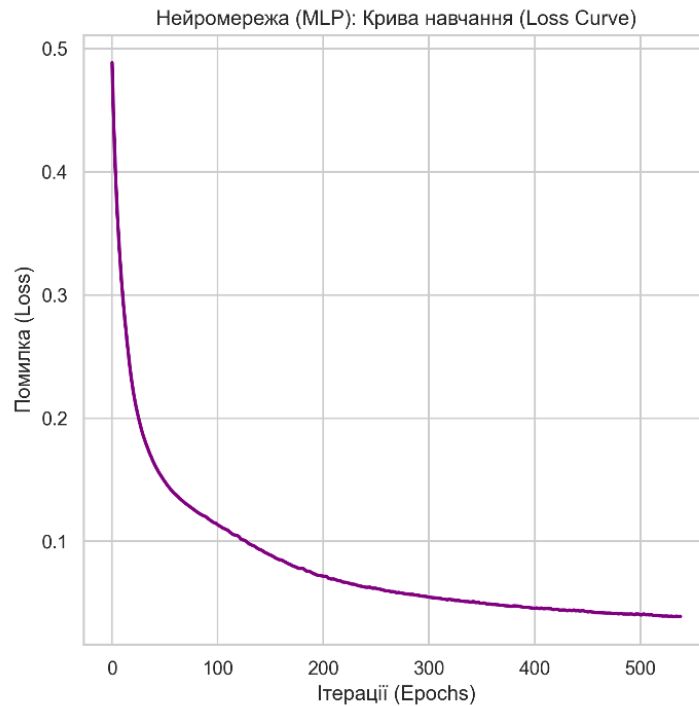


Рис. 2.2 Нейромережа (MLP). Крива навчання (Loss Curve)

Багатошаровий перцептрон (MLP) із функцією активації ReLU апроксимує нелінійну функцію f [13]:

$$y_{hat} = W_3 * ReLU(W_2 * ReLU(W_1 * x + b_1) + b_2) + b_3 \quad (2.8)$$

де W_i , b_i є матрицями ваг і векторами зміщення відповідних шарів нейронної мережі. Перед подачею до перцептрону (MLP) вхідні дані проходять стандартизацію:

$$x_{norm} = \frac{x - \mu}{\sigma} \quad (2.9)$$

де μ і σ є вибірковими середнім і стандартним відхиленням по кожній ознаці тренувальної вибірки.

2.4 Математична модель RL-системи

Задачу DDA в термінах навчання з підкріпленням формалізовано як дискретний марковський процес прийняття рішень (MDP). Для системи розроблено два незалежні MDP, що визначають поведінку різних агентів. У марковському процесі прийняття рішень (MDP) майбутній стан середовища залежить лише від поточного стану і вибраної агентом дії, але не від попередньої історії взаємодій.

2.4.1 MDP для штучного гравця BotAgent

Простір стану S_{bot} включає вектор спостережень розмірністю приблизно 103 дійсних чисел (float). До нього входять: дані 14-променевого конуса сенсорів (RayPerceptionSensor3D, візуалізовано на рис. 2.3), нормалізований вектор швидкості, логічний прапорець контакту з землею (isGrounded), час утримання стрибка (jumpHoldTime), когнітивного навантаження (cognitiveLoad), а також нормалізований вектор і відстань до наступної контрольної точки [30].

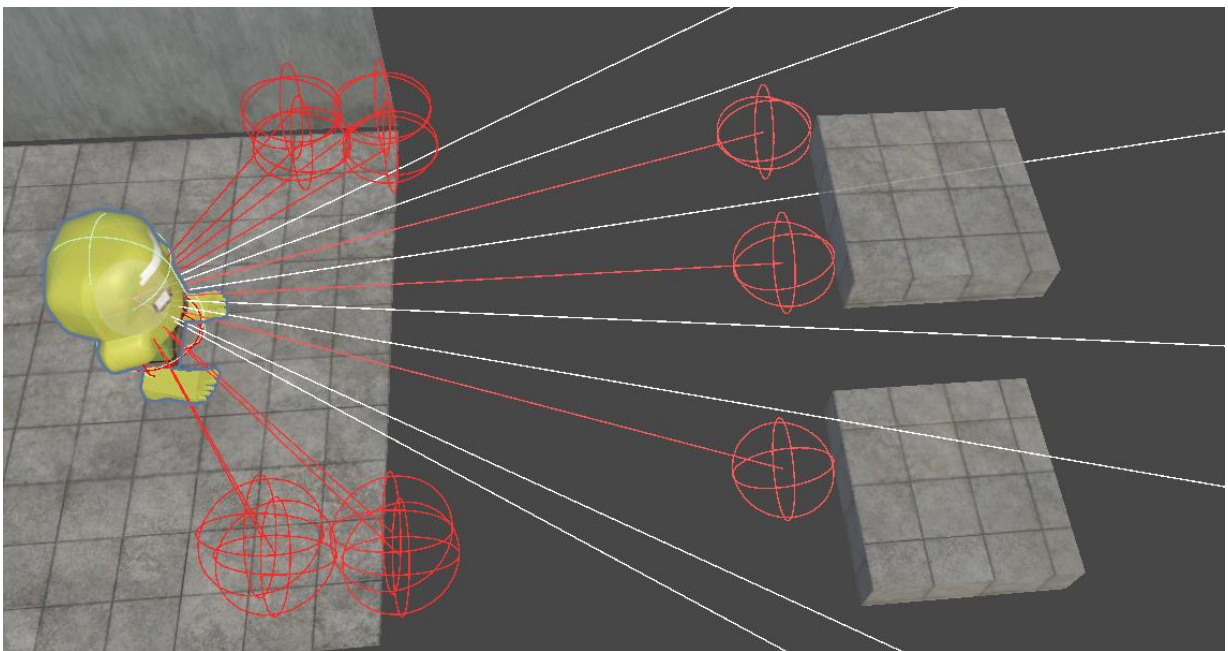


Рис. 2.3 Візуалізація сканування перешкод за допомогою променевого сенсору

Простір дій A_{bot} включає дві дискретні гілки: переміщення з 5 варіантами (стоїть, вперед, назад, вліво, вправо) і стрибка з 2 варіантами (кнопка відпущена, кнопка натиснута). Варіативний стрибок реалізується природним чином: агент

посилає команду стрибка кілька кадрів підряд, що відповідає тривалому утриманню кнопки у грі.

Для BotAgent запропоновано функцію винагороди, яка обчислюється так:

$$r_{\text{bot}}(t) = 0.005 * \max(0, \text{dot}(v_{\text{norm}}, d_{\text{checkpoint}})) - 0.001 + r_{\text{event}} \quad (2.10)$$

де v_{norm} – нормалізована швидкість, $d_{\text{checkpoint}}$ – одиничний вектор до контрольної точки, r_{event} дорівнює +1.0 при досягненні цілі, -1.0 при смерті, та +0.2 при успішному подоланні перешкоди.

2.4.2 MDP для режисера (DirectorAgent)

Простір стану S_{dir} складається з 13 агрегованих поведінкових метрик стану бота та ковзного вікна з 10 перешкод ($10 * 9 = 90$ значень):

$$S_{\text{dir}} = [\text{bot}_{\text{metrics}}(13), \text{obsatcle}_{\text{window}}(10 * 9)] \quad (2.11)$$

Кожна перешкода у вікні описується 3D-позицією відносно бота, унітарним кодуванням (one-hot) її типу та поточним множником складності. Ковзне вікно охоплює 2 перешкоди позаду і 8 попереду відносно поточної позиції бота.

Простір дій A_{dir} містить 12 неперервних значень у діапазоні $[-1, 1]$:

- дельти множників для 10 перешкод у вікні;
- дельта рівня атмосферного напруження;
- дельта рівня звукового впливу.

Функція нагороди Режисера побудована на базі гауссівської цільової функції [8] (рис. 2.4):

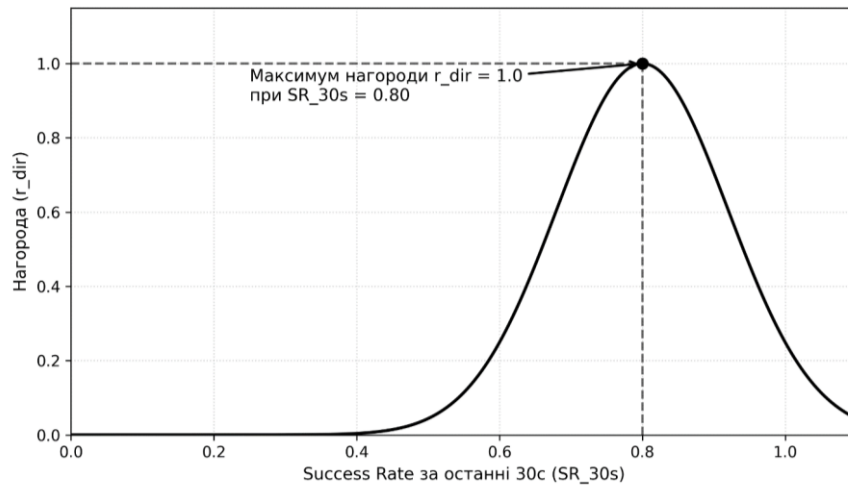


Рис. 2.4 Графік функції нагороди режисера

$$r_{dir}(t) = e^{-\frac{(SR_{30s}-0.80)^2}{2*0.12^2}} \quad (2.12)$$

де SR_{30s} – частка успішно пройдених перешкод у ковзному вікні останніх 30 секунд. Функція досягає максимуму 1.0 при $SR = 0.80$ (емпіричне “правило 80 відсотків”) і спадає до нуля при екстремально низьких чи високих значеннях. Додаткові штрафи розраховуються за формулою:

$$r_{dir} = r_{dir} - 0.5 * I(stuck_{30s}) - 0.3 * I(death_{10s} > 5) - 0.1 * I(idle > 5) \quad (2.13)$$

де $I(.)$ є індикаторною функцією відповідних умов фрустрації (29).

Після формалізації математичних моделей трьох DDA-підходів переходимо до архітектурного опису, що визначає, як ці моделі інтегруються у єдину систему

2.5 Архітектурні моделі та UML-діаграми системи

2.5.1 Діаграма класів

Архітектуру системи побудовано з використанням патерну «Одинак» (Singleton) для центральних менеджерів. DDAManager підписується на глобальні події GameEvents та делегує обчислення обраних стратегій IDDAStrategy.

Відповідну діаграму для системи DDA наведено в розділі графічних ілюстрацій на аркуші 4.

Конкретні реалізації `HeuristicDDAStrategy` і `MachineLearningDDAStrategy` поліморфно замінюють одна одну без змін у базовому класі. `DDAIferenceManager` відповідає виключно за виведення (інференс) і підтримує два механізми: згенерований `RandomForestModel` (статичний C#-клас) і `Unity Worker (ONNX)`.

Ієрархія перешкод будується навколо абстрактного класу `DynamicObstacleBase`, що реалізує `IDynamicObstacle`. Базовий клас інкапсулює всю логіку буферизації (метод `AdjustDifficulty`) і захисту від відкату (`ddaProgressTracker`). Конкретні перешкоди визначають лише метод `ApplyDDA(float multiplier)`, що відповідає принципу єдиної відповідальності.

2.5.2 Діаграма послідовності

Типовий життєвий цикл обробки ігрової події (на прикладі виявлення кластера смертей) наведено на графічній ілюстрації аркуша 5. Після загибелі гравця модуль `PlayerTracker` фіксує позицію. `GameSession` записує ці дані і викликає перевірку кластеризації. Якщо кластер виявлено, система генерує подію, яку перехоплює активна стратегія. Після обчислення нового множника перешкода буферизує команду і очікує події затемнення екрана (`OnScreenDarkened`), після якої фізично застосовує оновлені параметри.

2.5.3 Структура даних телеметрії

Система аналітики формує дані у форматі NDJSON (Newline-Delimited JSON), де кожен рядок є самостійним незалежним JSON-об'єктом. Це дозволяє здійснювати потоковий запис в кінець файлу, без перезапису і ризику пошкодження файлу в разі критичної помилки гри.

Структура запису події загибелі має такий вигляд:

```
{
  "session_id": "9a2c827633a94333b6d9e6a15544da43",
  "user_id": "75afef04b813497185d1aad8925e5a06",
  "timestamp_iso": "2026-05-26T21:01:36.2417154Z",
  "event_type": "death",
  "event_seq": 26,
  "event_data":
  {
```

```

        "death_id":1,
        "level":0,
        "cause":"fall",
        "time_spent_on_level":35.599987,
        "total_idle_time":0.0,
        "camera_jitter":36.61169,
        "input_rate":0.535942256,
        "avg_time_to_recover":0.0,
        "position":[196.215729,16.6743355,-8.253137]
    }
}

```

Рядок ручного зворотного зв'язку, що виступає як маркований навчальний приклад для ML-моделі, містить наступний вектор ознак:

```

{
    "session_id":"5087e62da529499ab3c7895d43f1fe81",
    "user_id":"75afef04b813497185d1aad8925e5a06",
    "timestamp_iso":"2026-05-26T19:05:39.7981009Z",
    "event_type":"player_feedback",
    "event_seq":190,
    "event_data":
    {
        "level":3,
        "label":"Manual Tuning",
        "desired_multiplier":0.5028162,
        "deaths_at_current_checkpoint":0,
        "recent_cluster_deaths":1,
        "time_spent_on_level":43.20819,
        "avg_time_to_recover":0.2546997,
        "total_idle_time":0.395782471,
        "camera_jitter":0.0,
        "current_input_rate":0.817534268,
        "position":[103.070824,19.72,-3.675403]
    }
}

```

Висновки до розділу 2

У другому розділі сформульовано та деталізовано комплекс технологічних і функціональних вимог до системи. Розроблено математичну модель евристичної адаптації, що інтегрує алгоритми просторової кластеризації смертей, обчислення центроїда і зваженого впливу (формули (2.1) – (2.5)).

Визначено вектор ознак із восьми метрик для машинного навчання і математично формалізовано задачу прогнозування складності як задачу регресії з учителем (формули (2.6) – (2.9)). Описано архітектуру марковського процесу прийняття рішень (MDP) для обох RL-агентів, включаючи запропоновану гауссівську цільову функцію винагороди Режисера, що кодує оптимальну залученість на рівні 80% (формули (2.10) – (2.13)).

За допомогою UML-діаграм класів та послідовності обґрунтовано архітектурні рішення і потоки даних, що гарантують розширюваність системи та невидимість втручання алгоритмів адаптації у процес гри. Встановлено формат збереження телеметрії (NDJSON), придатний для подальшої обробки у ML-конвеєрі, із прикладами структур рядків подій.

3. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ІНТЕГРАЦІЯ МАШИННОГО НАВЧАННЯ

3.1 Архітектура системи і принципи SOLID

Проектована система складається з близько 30 класів та інтерфейсів, розподілених у п'яти просторах імен (namespaces). Архітектурні рішення свідомо орієнтовані на дотримання принципів SOLID, що є необхідною умовою для забезпечення розширюваності та можливості заміни алгоритмів DDA без переписування коду ігрових об'єктів та перешкод. Особливості застосування принципів SOLID у проєкті наведено у таблиці 3.1.

Таблиця 3.1

Застосування принципів SOLID у проєкті

Принцип	Реалізація
SRP	GameSession відповідає лише за агрегацію метрик; AnalyticsManager лише за запис; DDAManager лише за маршрутизацію
OCP	Інтерфейс IDDAStrategy дозволяє додавати нові стратегії без зміни DDAManager
LSP	HeuristicDDAStrategy і MachineLearningDDAStrategy є взаємозамінними реалізаціями IDDAStrategy
ISP	Окремі інтерфейси IDynamicObstacle, IDynamicEntity, IDynamicAtmosphere, IDynamicAudio
DIP	DDAManager залежить від абстракції IDDAStrategy, не від конкретних класів стратегій

Патерн Спостерігач (Observer) реалізовано через статичний клас GameEvents із використанням делегатів Action<T> мови C#. Це забезпечує слабке зв'язування між компонентами: підсистема GameSession не знає про існування DDAManager, а DDAManager не володіє інформацією про те, яка саме перешкода зреагує на подію адаптації.

Патерн Реєстр (Registry) реалізовано через статичну колекцію HashSet<IDynamicObstacle> у класі DDAManager. Перешкоди самотійно реєструються під час активації (OnEnable) і видаляються при деактивації

(OnDisable), що виключає помилки втрачених посилань (NullReferenceException) при переході між сценами.

3.2 Реалізація ігрового ядра

3.2.1 Контролер персонажа

Клас PlayerMovement реалізує фізичний контролер персонажа на базі компонента Rigidbody з вимкненим фізичним обертанням. Вектор руху обчислюється відносно системи координат камери, що забезпечує інтуїтивне керування незалежно від кута повороту персонажа:

$$V_{move} = input_y * cam_{forward} * v_{speed} + input_x * cam_{right} * h_{speed}$$

Підтримується механіка “Час койота”: короткий часовий буфер після втрати контакту з краєм платформи, протягом якого стрибок усе ще можливий. Значення *coyoteTime* = 0.1 с підбиралось емпірично як мінімально необхідне для комфортного керування. Механіка варіативного стрибка реалізується через динамічну зміну прискорення: якщо гравець відпустив кнопку стрибка раніше за досягнення максимальної висоти траєкторії, застосовується додаткове гравітаційне прискорення *shortJumpMultiplier*.

Зовнішній імпульс (наприклад, від платформи, що відштовхує) зберігається у окремому векторі *externalVelocity* і поступово згасає за функцією *MoveTowards* зі швидкістю *externalVelocityDecay*. У момент торкання землі цей вектор обнуляється миттєво.

3.2.2 Система відродження і TransitionManager

Клас *RespawnSystem* зберігає поточну точку відродження і обробляє виклик *Respawn()*. За наявності *TransitionManager* відродження відбувається у три фази: запуск візуальної анімації *FadeToBlack* (затемнення), миттєве переміщення (телепортація) та скидання фізичних імпульсів у момент, коли екран повністю чорний (функція зворотного виклику *onMidpoint*), та подальше освітлення екрана *FadeFromBlack*. Саме у фазу *onMidpoint* клас *DDAManager* отримує подію *OnScreenDarkened* і фізично застосовує всі накопичені зміни до перешкод. Саме це становить технічну основу механіки непомітної для гравця

адаптації [27]. Для наочного розуміння послідовності цих процесів, спрощену схему життєвого циклу адаптації (від моменту смерті гравця до видимої зміни перешкоди) наведено на рис. 3.1.



Рис. 3.1 Спрощена схема життєвого циклу адаптації

3.2.3 PlayerTracker і LastSafePosition

Клас `PlayerTracker` відстежує останню безпечну позицію гравця (`LastSafePosition`), яка оновлюється у методі `FixedUpdate`, лише за умови гравця на поверхні `isGrounded == true`. Ця координата використовується під час обчислення центроїда просторового кластера замість фактичної позиції смерті гравця (площини `KillPlane`). Практичне значення цього підходу полягає в наступному: якщо гравець падає з платформи на висоті 15 м у прірву на позначці -5 м, координата самої смерті є неінформативною, оскільки всі смерті акумулюються на одній висоті. Натомість `LastSafePosition` вказує на точне місце стрибка і ідентифікує проблемну перешкоду.

3.3 Підсистема збору телеметрії

`GameSession` є класом, реалізованим за патерном «Одинак» (`Singleton`) із властивістю `DontDestroyOnLoad`, що гарантує його збереження між ігровими сценами для неперервної агрегації метрик сесії. Клас реалізує шість груп вимірювань.

Вимірювання смертей. Метод `RegisterDeath` додає новий запис `DeathRecord` до кільцевого буфера `recentDeaths` фіксованого розміру і видаляє записи, старші за визначений проміжок часу. Після кожного запису викликається

метод перевірки кластерів, що перевіряє наявність локалізованої проблеми за критеріями відстані та мінімальної кількості смертей.

Вимірювання бездіяльності. У методі CheckIdle відстежується наявність будь-якого вводу від миші або клавіатури. Якщо ввід відсутній довше за встановлений поріг, фіксується початок сесії бездіяльності (Idle Session). Після відновлення активності тривалість сесії додається до загальної змінної totalIdleTimeThisLevel.

Вимірювання APM. У часовому вікні behavioralWindowSize (в секундах) підраховуються натискання клавіш і кнопок миші. Поточна частота розраховується як відношення кількості натискань до часу, що минув від початку вікна.

Вимірювання тремтіння камери (Jitter). Показник обчислюється як кумулятивна сума модуля переміщення курсора або камери за кадр, якщо таке переміщення перевищує визначений поріг “паніки” у пікселях:

$$J = J + \text{delta} * dt, \quad \text{delta} > j_{\text{thresh}} \quad (3.1)$$

Вимірювання часу відновлення. Після кожного відродження фіксується внутрішній час і встановлюється прапорець isRecovering. При першому вводі від клавіатури або миші прапорець знімається, а різниця в часі додається до показника.

Менеджер аналітики (AnalyticsManager) реалізує асинхронний буферний запис: події накопичуються у черзі Queue<string> і записується на диск при заповненні буфера (наприклад, 50 рядків), або під час примусового скидання ForceFlush() (при виході з гри). Запис відбувається у окремому потоці за допомогою механізму C# async/await, що виключає блокування головного ігрового циклу.

3.4 Реалізація евристичної DDA-системи

3.4.1 HeuristicDDAStrategy

Клас HeuristicDDAStrategy реалізує інтерфейс IDDAStrategy і містить два публічні методи. Метод обробки застрягання гравця обробляє кластер смертей: для кожної перешкоди у множині обчислюється відстань до координати проблеми, на її основі розраховується ваговий коефіцієнт (згідно з формулою (2.4)) і викликається метод AdjustDifficulty у буферизованому режимі. Аналогічно працює метод обробки домінування гравця, але у значно ширшому радіусі і зі знижувальним множником ($M < 1.0$). Динаміку зміни складності та симуляцію стану "потoku" евристичною моделлю наведено на рис. 3.2.

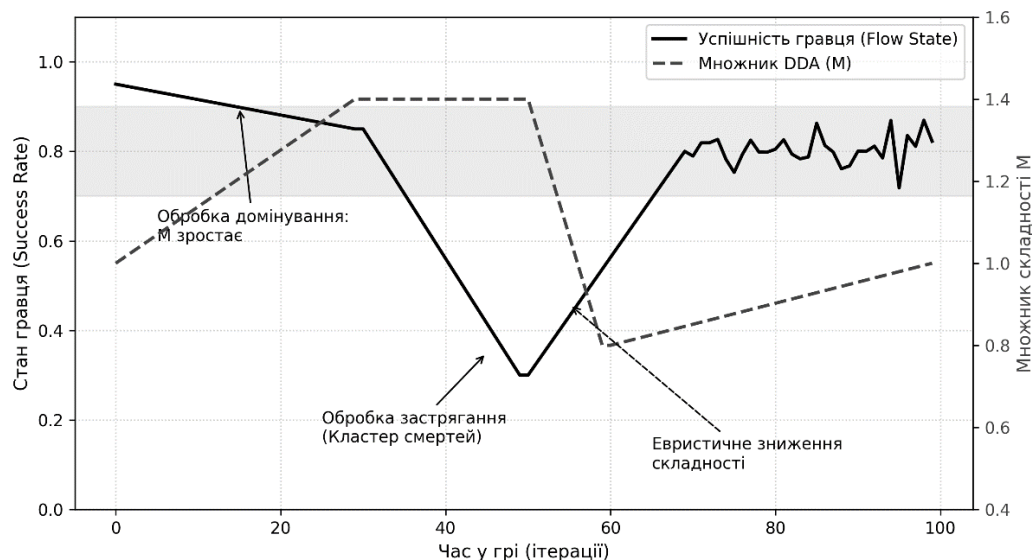


Рис. 3.2 Симуляція потоку та динаміка множника в евристичній системі DDA

3.4.2 DynamicObstacleBase

Метод AdjustDifficulty(float multiplier, AdjustmentMode mode) є єдиним публічним інтерфейсом управління складністю перешкоди. Метод виконує чотири завдання: по-перше, обчислює зважений множник з урахуванням базової ваги перешкоди; по-друге, виконує перевірку надійності – ігнорує виклик, якщо нове значення ускладнює гру, але попередній максимум полегшення (ddaProgressTracker) ще не вичерпано; по-третє, оновлює цільові показники; по-

четверте, застосовує зміни негайно або очікує на подію затемнення екрана. Конкретна перешкода визначає виключно метод ApplyDDA.

Візуальний приклад впливу різних значень множника складності на стан динамічної перешкоди (на прикладі простої платформи) наведено на рис. 3.3.



Рис. 3.3 Перешкода до і після адаптації (візуалізація станів при множниках 0.65, 1.0 та 1.35)

Детальні правила та параметри адаптації для всіх реалізованих типів перешкод систематизовано в таблиці 3.2.

Таблиця 3.2

Параметри DDA для різних типів перешкод

Тип	ApplyDDA: multiplier > 1	ApplyDDA: multiplier < 1
SimpleDynamicPlatform	Збільшення X, Z (ширша платформа)	Зменшення X, Z (вужча)
MovingPlatform	Зменшення швидкості руху	Збільшення швидкості
FallingPlatform	Збільшення часу до падіння	Зменшення часу
SpikyPlatform	Збільшення паузи між шипами	Зменшення паузи

ML-моделі вимагають заздалегідь зібраних даних з людською розміткою. Наступний підрозділ описує реалізацію RL-підходу, що усуває цю залежність

3.5 ML-конвеєр та навчання моделей

3.5.1 Збір та розвідувальний аналіз даних (EDA)

Набір даних (датасет) формується в режимі DDAMode. DataCollection_Baseline. Тестувальники проходять рівні у звичайному режимі (DDA вимкнено), але система зворотного зв'язку дозволяє їм фіксувати суб'єктивне відчуття складності. Натискання визначеної клавіші "Занадто складно" або "Занадто легко" генерує подію, яка записує в журнал повний вектор із 8 поведінкових ознак та відповідну мітку бажаного множника `desired_multiplier`.

Для розуміння природи зібраних даних було проведено розвідувальний аналіз (Exploratory Data Analysis). На рис. 3.4 наведено матрицю кореляції ознак, яка демонструє взаємозв'язок між психологічними показниками та реальною складністю.

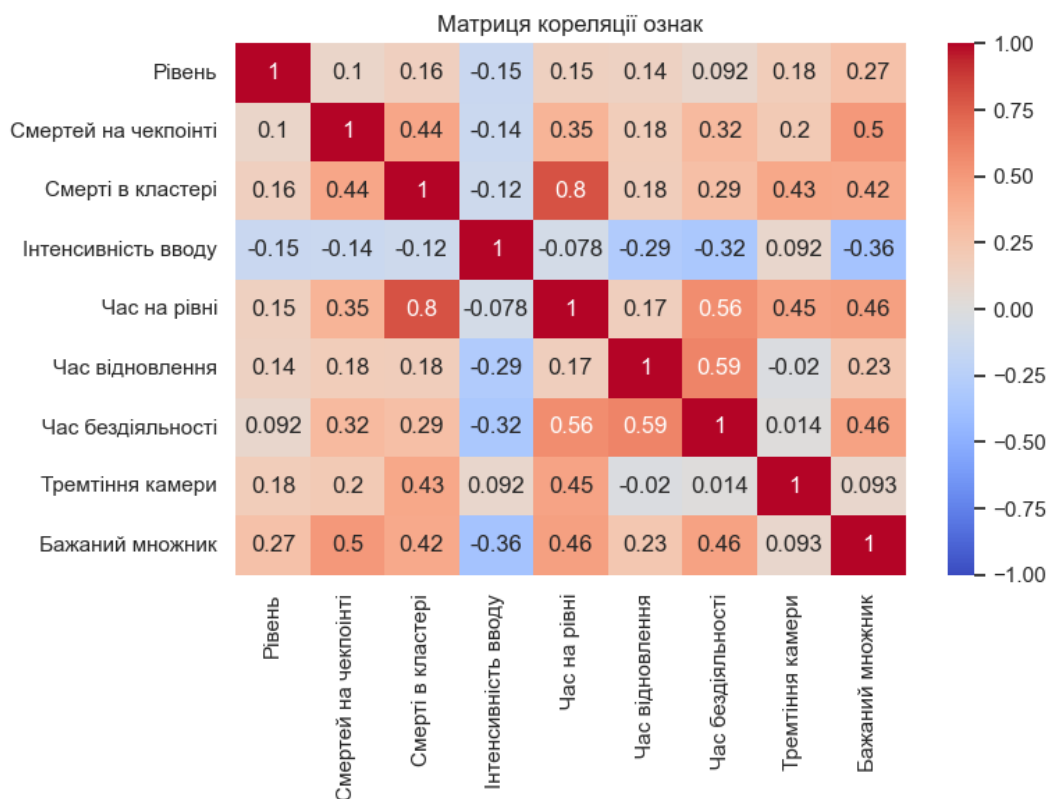


Рис. 3.4 Матриця кореляції поведінкових ознак гравця

Як видно з графіка попарного розподілу психологічних ознак (рис. 3.5), метрики часу відновлення та тремтіння камери формують чітко відокремлені

кластери у випадках сильної фрустрації гравця. Загальний розподіл зібраних бажаних множників та залежність кількості смертей від множника наведено на рис. 3.6 та рис. 3.7 відповідно.

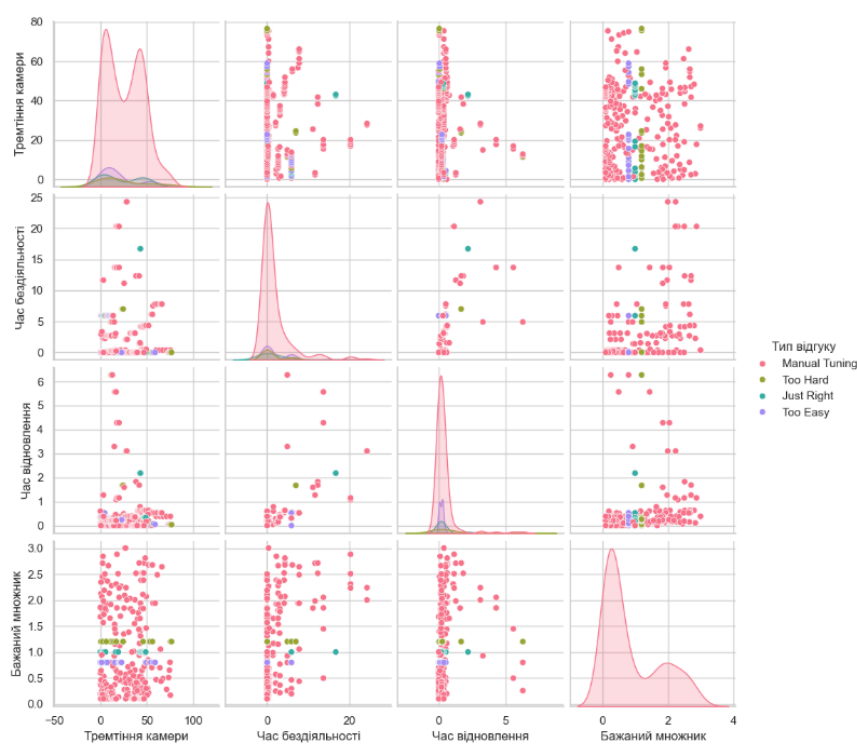


Рис. 3.5 Парний розподіл психологічних метрик (Pairplot)

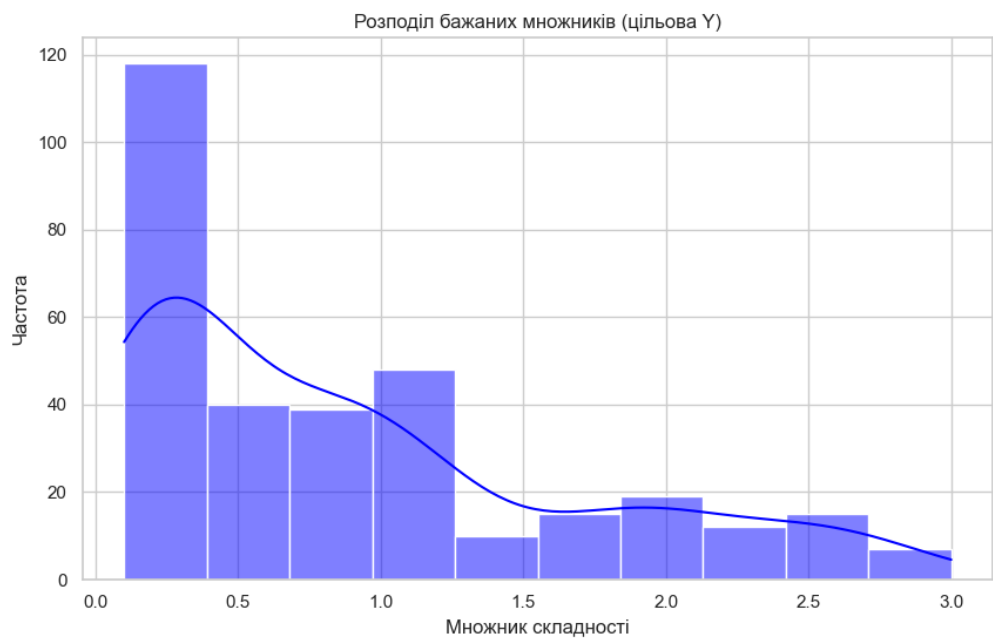


Рис. 3.6 Загальний розподіл бажаних множників

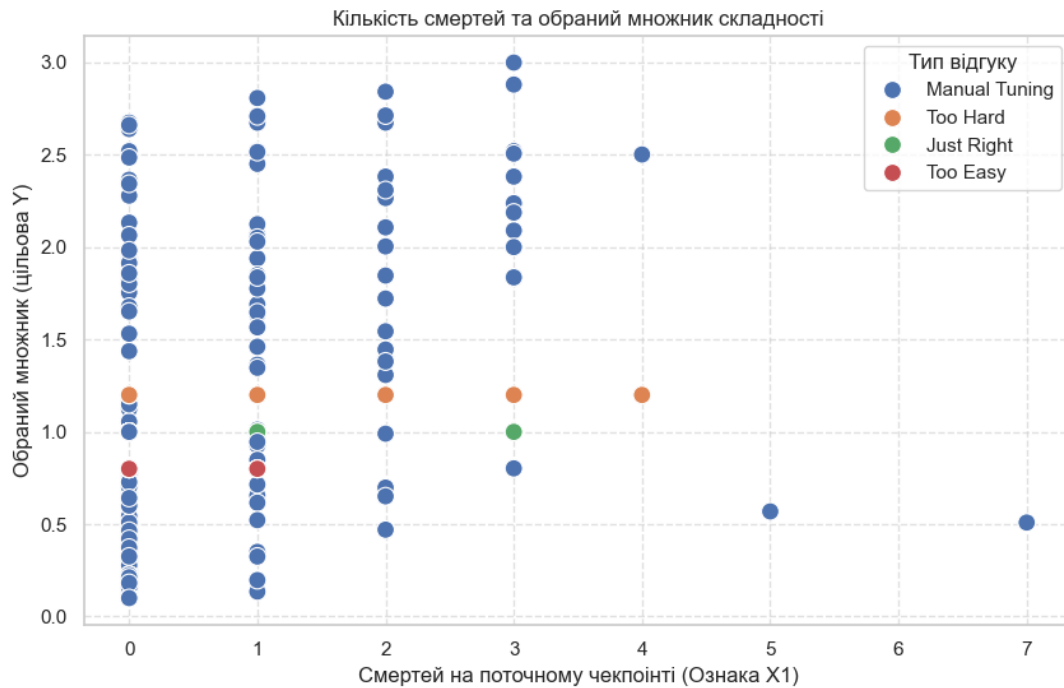


Рис. 3.7 Залежність кількості смертей від множника

Загальний конвеєр підготовки даних виконується скриптом Python: завантаження файлів NDJSON, усунення дублікатів (дедублікація), виділення маркованих рядків `player_feedback`, форматування вектора x та мітки y , і подальше розбиття на тренувальну та тестову вибірки у пропорції 80:20.

3.5.2 Навчання моделі **Random Forest**

Модель регресії випадкового лісу (`RandomForestRegressor` з бібліотеки `scikit-learn`) навчається з такими гіперпараметрами: `n_estimators = 200`, `max_depth = 12`, `min_samples_leaf = 2`. Структуру одного з дерев рішень в ансамблі наведено на рис. 3.8.

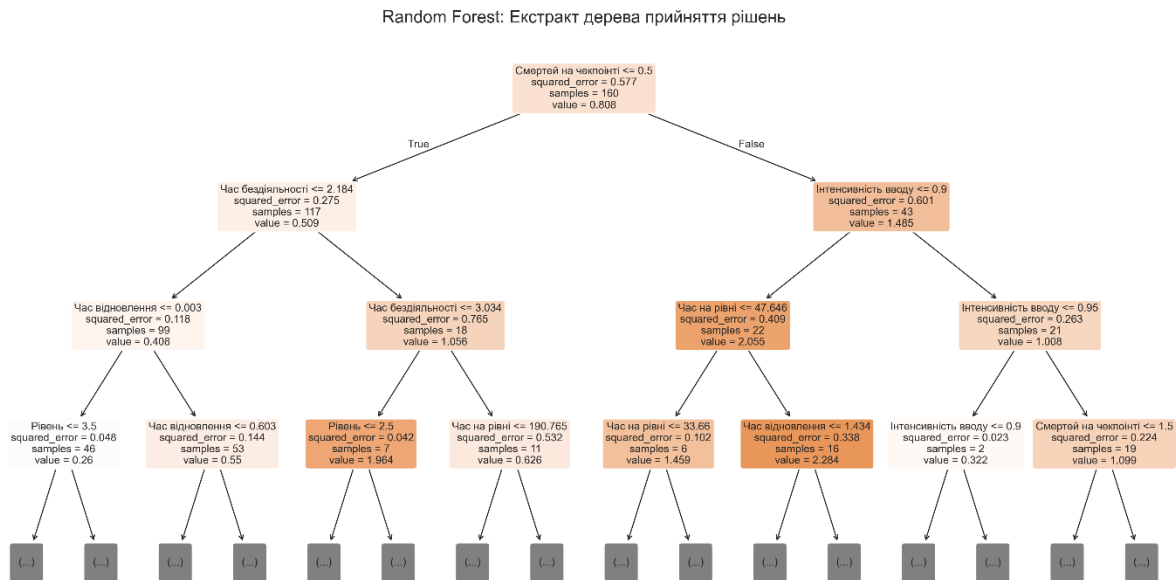


Рис. 3.8 Візуалізація структури прийняття рішень одним деревом у Random Forest

Для оцінки впливу кожної метрики на прийняття рішення системою було побудовано графік важливості ознак (Feature Importances) (рис. 3.9) та проведено SHAP-аналіз (рис. 3.10), який показав технічну вагомість психологічних метрик у прогнозуванні.

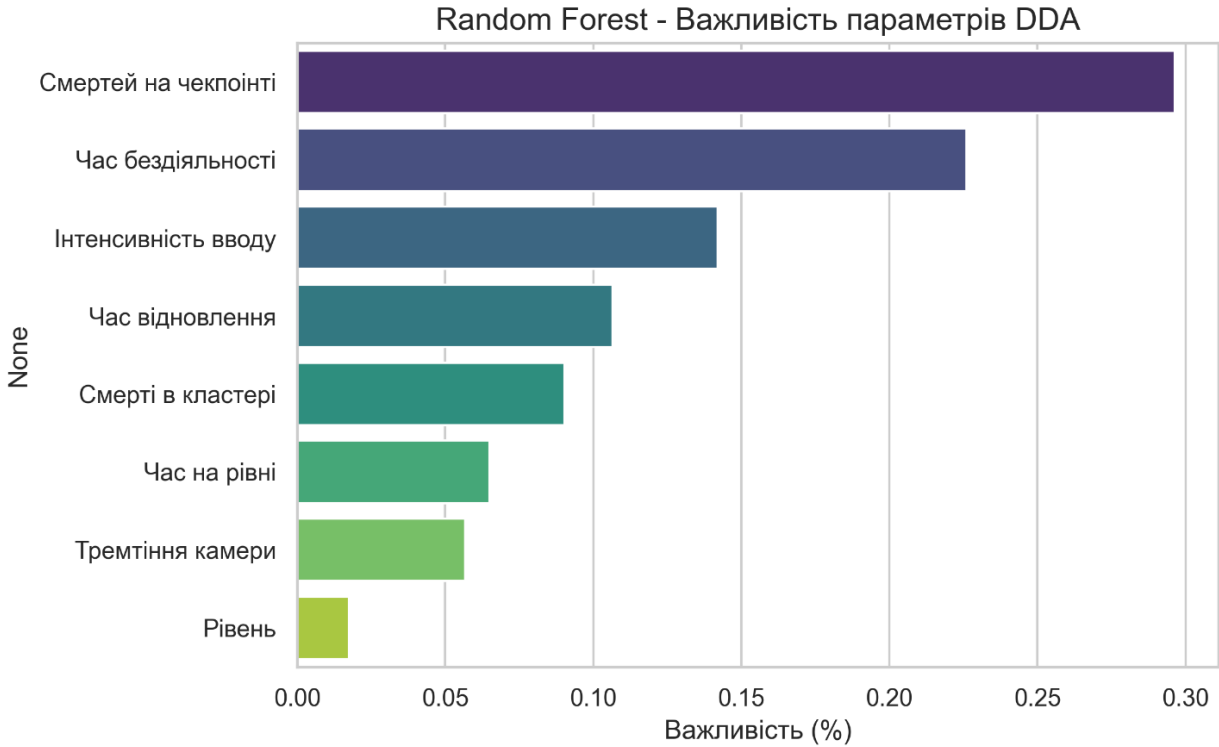


Рис. 3.9 Графік важливості ознак для моделі Random Forest

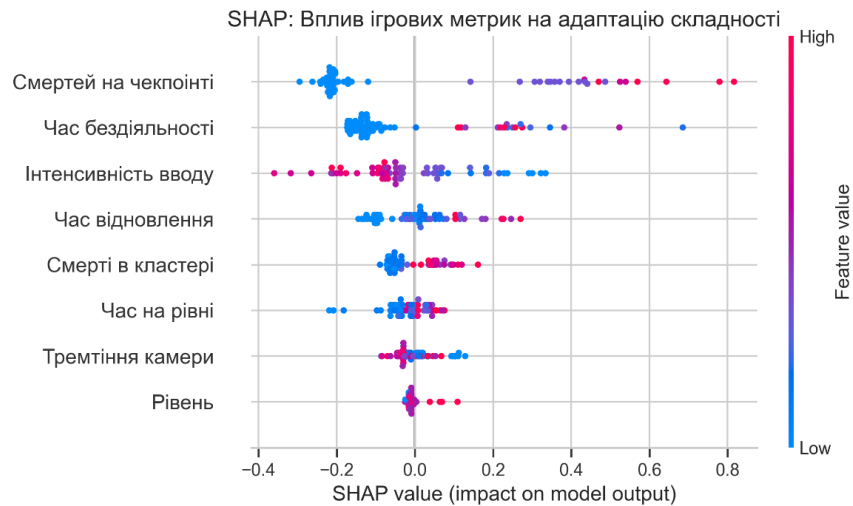


Рис. 3.10 SHAP-аналіз впливу ознак на вихід моделі

Якість прогнозування на тестовій вибірці проілюстровано графіком залежності фактичних та передбачених значень (рис. 3.11). У такій візуалізації ідеальний прогноз відповідає діагональній лінії, тому чим щільніше точки розташовані навколо неї, тим вищою є точність моделі. Після навчання виконується 5-кратна перехресна перевірка (Cross-Validation). Навчена модель транслюється в мову C# за допомогою інструменту m2scgen і зберігається як статичний скрипт без сторонніх залежностей.

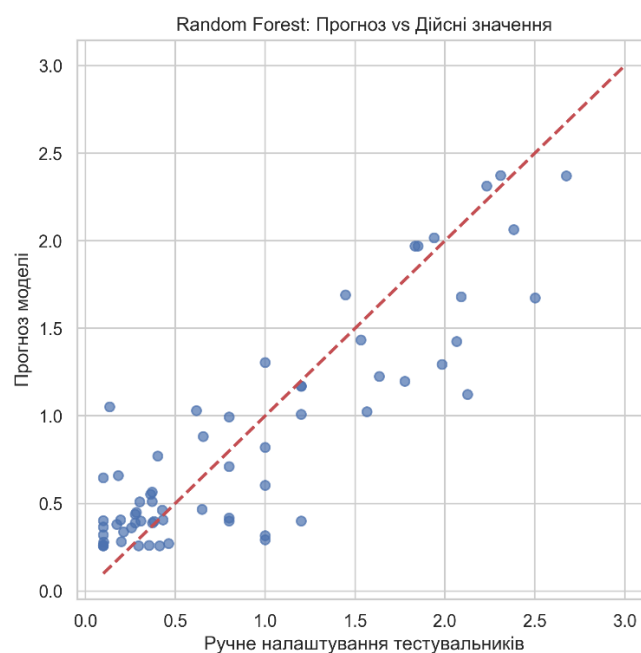


Рис. 3.11 Графік фактичних і передбачених значень для Random Forest

3.5.3 Навчання MLP і конвертація у ONNX

Багатошаровий перцептрон навчається за допомогою MLPRegressor з двома прихованими шарами (64 і 32 нейрони), активацією ReLU та оптимізатором Adam (до 1000 ітерацій). Динаміку процесу навчання відображає графік кривої навчання (Learning Curve) (рис. 3.12).

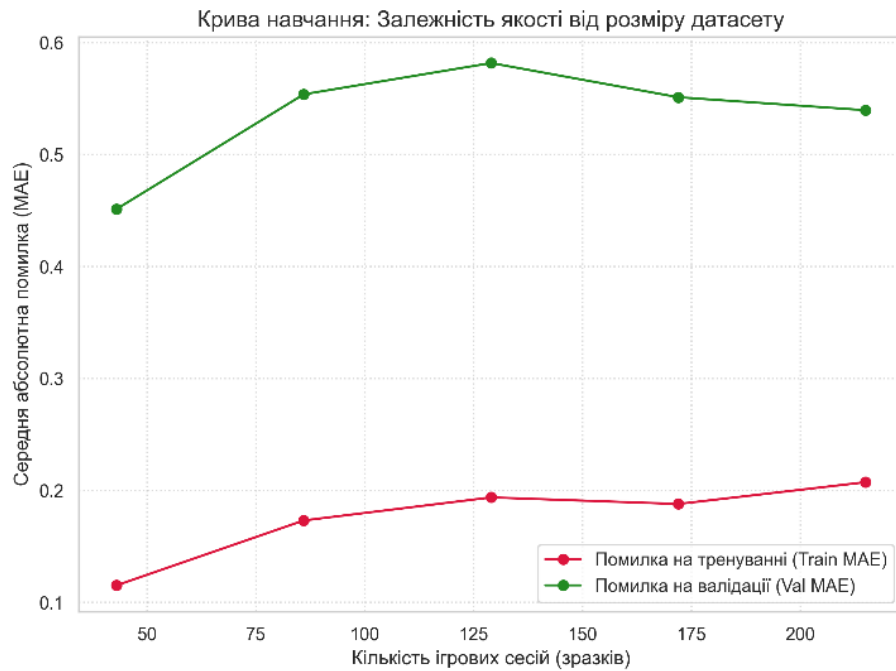


Рис. 3.12 Крива навчання багатошарового перцептрону

Оцінка відхилень та похибок моделі MLP проводилася за допомогою аналізу залишків (Residual Plot) (рис. 3.13). У цьому методі ідеальним показником є симетричне та щільне розташування точок (помилки) навколо горизонтальної нульової осі: чим ближче відхилення до нуля, тим вищою є якість передбачень моделі. Додатково було досліджено графік часткової залежності (Partial Dependence Plot) (рис. 3.14) для метрик Jitter, смертей на точці збереження та загальному часу на рівні.

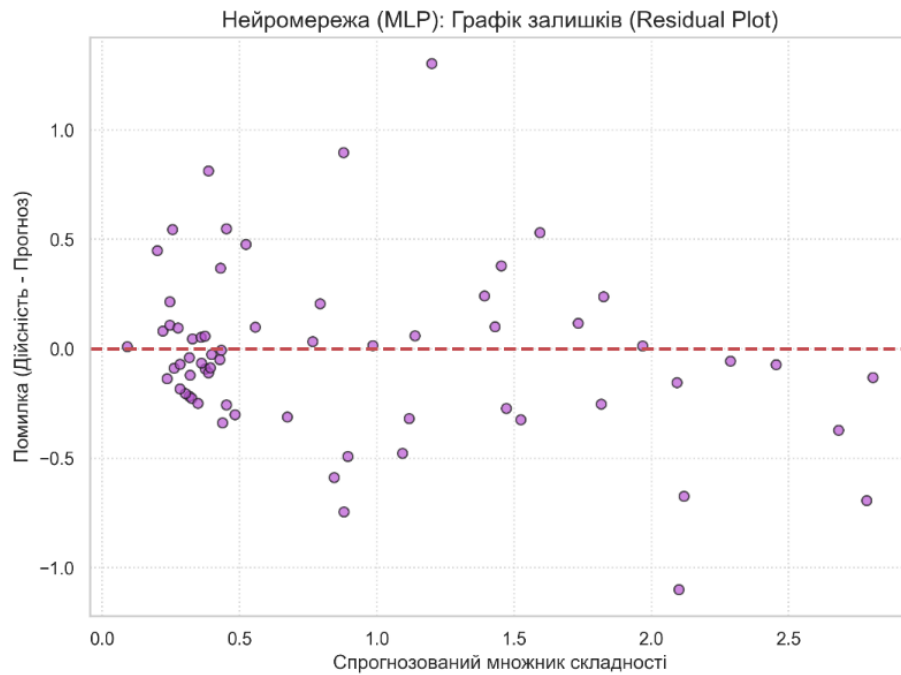


Рис. 3.13 Графік аналізу залишків (Residual Plot) моделі MLP

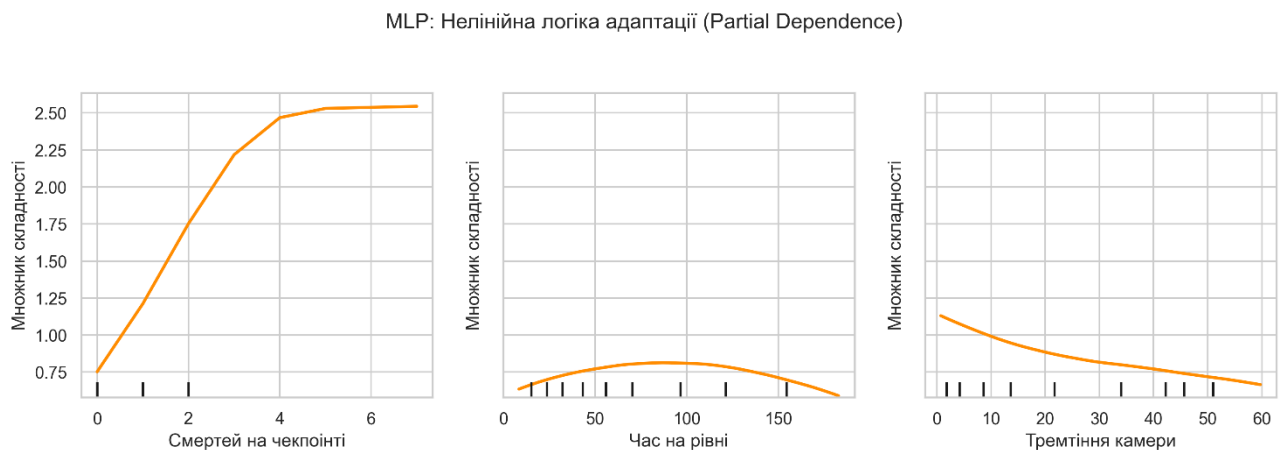


Рис. 3.14 Графік часткової залежності для метрик

Для використання нейронної мережі в ігровому рушії виконується її конвертація у формат ONNX за допомогою `skl2onnx` з розміром пакета даних (batch size), що дорівнює 1. У середовищі Unity виведення (інференс) моделі ONNX забезпечується пакетом Unity Sentis. Вираз `using Tensor<float> inputTensor = new Tensor<float>(...)` гарантує миттєве звільнення пам'яті (Garbage Collection) одразу після отримання результату, зберігаючи високу продуктивність.

3.6 Реалізація RL-системи

3.6.1 Граф рівня (LevelGraph) і ковзне вікно перешкод

Клас LevelGraph сканує сцену при старті й збирає всі об'єкти із компонентом Obstacle. Числовий індекс витягується з кінця імені об'єкта. Відсортований масив є топологічним графом рівня. Метод отримання вікна перешкод повертає масив заданої довжини, використовуючи доповнення нулями на краях масиву. Індекс поточної перешкоди оновлюється при кожному успішному проходженні; при відродженні відновлюється до індексу, збереженого активною точкою збереження.

3.6.2 Штучний гравець (BotAgent)

Клас BotAgent є похідним від Unity.MLAgents.Agent. У фазі збору спостережень (CollectObservations) до сенсора подається вектор внутрішніх станів: нормалізована швидкість, час утримання стрибка, когнітивне навантаження тощо. Разом з даними від системи візуальних променів (RayPerceptionSensor3D) розмірність вектора спостережень перевищує 90 значень. Функція нагородження стимулює рух до цілі та штрафує за падіння (згідно з формулою (2.10)).

3.6.3 Агент-режисер (DirectorAgent)

Клас DirectorAgent керує параметрами ігрового світу. Період прийняття рішень (DecisionPeriod) встановлено на рівні 50 кроків симуляції, що відповідає частоті один раз на секунду (при 50 FPS), оскільки Режисер здійснює макроуправління рівнем. Агент генерує 12 неперервних дій: 10 з них регулюють складність найближчих перешкод, а дві відповідають за атмосферну та звукову напруженість гри.

3.6.4 Фази тренування

Тренування засновано на методології асиметричної самогри і складається з двох етапів. Динаміку тренування агентів ботів (зростання нагороди, зміна довжини епізоду) збережено та візуалізовано за допомогою TensorBoard (рис. 3.15).

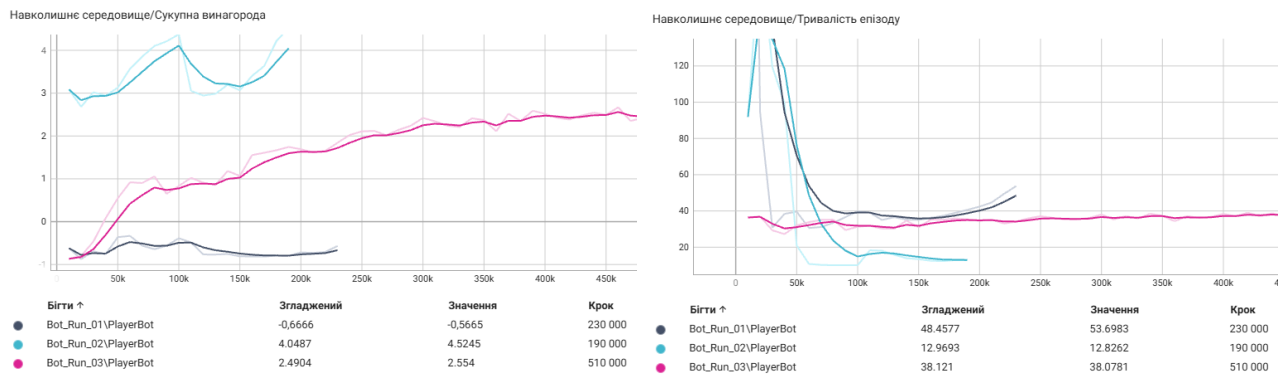


Рис. 3.15 Графіки тренування трьох агентів ботів у TensorBoard: сукупна нагорода та тривалість епізодів

Фаза 1 (Навчання Бота). Запускається алгоритм PPO у середовищі зі статичною складністю. Цільовим показником є досягнення ботом стабільного проходження рівня з імовірністю понад 80% за кілька мільйонів кроків симуляції. Отримана навчена модель експортується у формат ONNX.

Фаза 2 (Навчання Режисера). Модель штучного гравця завантажується в BotAgent як заморожена (її ваги більше не оновлюються). DirectorAgent, також через алгоритм PPO починає тренування, орієнтуючись на максимізацію гауссівської функції залученості шляхом ускладнення чи полегшення середовища для замороженого бота [29]. Після завершення навчання Режисер готовий до взаємодії з живими гравцями.

Висновки до розділу 3

У третьому розділі детально описано програмну реалізацію підсистеми DDA. Доведено ефективність застосування принципів об'єктно-орієнтованого проєктування SOLID (зокрема, патернів «Стратегія», «Спостерігач», «Одинак»), що забезпечує відокремлення логіки адаптації від механік ігрових перешкод.

Описано технічну реалізацію неперервного збору поведінкової телеметрії, зокрема такі психологічні метрики, як показник активності дій за хвилину (APM) та тремтіння камери (Camera Jitter). Тестування конвеєра машинного навчання підтвердило його працездатність: розвідувальний аналіз продемонстрував високу кореляцію психологічних ознак із реальною складністю (рис. 3.4, рис.

3.5). Реалізація моделей Random Forest та MLP (рис. 3.11, рис. 3.12) дала змогу генерувати прогнози без звернення до зовнішніх Python-залежностей у середовищі виконання Unity (через інструмент m2cgen та рушій Sentis).

Окремо описано етапи тренування архітектури навчання з підкріпленням (RL). Впровадження техніки асиметричної самогри дозволило Режисеру навчитися адаптувати складність на базі симуляцій штучного гравця, ізолювавши процес від необхідності проведення тисяч ітерацій тестування з живими людьми.

4. ТЕСТУВАННЯ, А/В АНАЛІЗ ТА ОЦІНКА ЕФЕКТИВНОСТІ DDA СИСТЕМИ

4.1 Методика тестування та організація А/В-груп

Для об'єктивного порівняння чотирьох розроблених підходів до динамічної адаптації складності використовувалася методика А/В-тестування з незалежними групами. Оскільки залучення багатотисячної аудиторії виходить за межі формату кваліфікаційної роботи, було сформовано малу фокус-групу з 8 учасників (віком від 14 до 21 років) із різним попереднім ігровим досвідом. Кожен учасник проходив три рівні в одному режимі DDA з паузами між рівнями. Тривалість сесії складала 18–25 хвилин залежно від групи. Перед початком сесії учасникам не повідомлялось про застосовуваний DDA-режим (одностороннє сліпе тестування). Після завершення проводилось анкетування за 5-бальною шкалою Лайкерта з пунктами: 1) суб'єктивна складність, 2) задоволення від процесу, 3) помічені зміни складності, 4) бажання продовжити гру. Дані телеметрії зберігалися автоматично у форматі NDJSON для подальшого аналізу.

Учасників було випадковим чином розділено на чотири ізольовані групи (по 2 особи у кожній). Кожна група проходила ідентичні ігрові рівні, проте з різним фоновим режимом DDA:

- Група А (Baseline) – контрольна група, гра без адаптації (статична складність).
- Група В (Heuristic) – гра з увімкненим математичним евристичним DDA.
- Група С (Machine Learning) – гра з моделлю Random Forest.
- Група D (Reinforcement Learning) – гра з активним агентом-Режисером.

Для кількісної та якісної оцінки ефективності було визначено такі цільові метрики:

1. Completion Rate (CR) – частка успішно подоланих перешкод (у відсотках) від загальної кількості спроб за всі ігрові сесії групи.
2. Average Deaths (AD) – середня кількість смертей гравця за одну ігрову сесію.

3. Session Time (ST) – середній час сесії у хвилинах.
4. Invisibility Score (IS) – суб'єктивна оцінка непомітності змін складності за 5-бальною шкалою (дані збиралися через анкетування після сесії).
5. Flow Score (FS) – суб'єктивна оцінка стану залученості та задоволення від гри (від 1 – «дуже нудно/дуже складно» до 5 – «ідеальний баланс»).

Тестування проводилось на трьох підготовлених рівнях: Рівень 1 (лінійна структура з 3 типами перешкод і 10 контрольними точками), Рівень 2 (нелінійна структура із зонами підвищеної складності) та Рівень 3 (дизайнерський рівень з новими перешкодами). Для кращого розуміння просторової топології та ігрових механік, з якими стикалися учасники фокус-груп під час тестування, загальний вигляд типового ігрового середовища з підписами ключових елементів наведено на рис. 4.1.

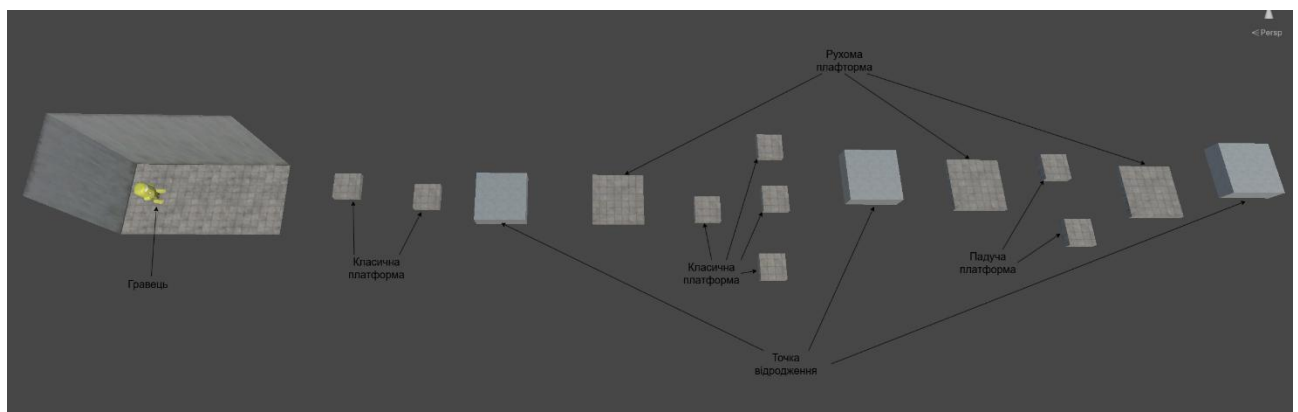


Рис. 4.1 Загальний вигляд фрагменту тестового рівня

4.2 Обмеження дослідження та загрози достовірності

Під час інтерпретації отриманих результатів слід враховувати такі методологічні обмеження та загрози достовірності.

Внутрішня достовірність. Учасники тестування знали, що беруть участь в експерименті, що могло вплинути на їхню поведінку (ефект Хоторна). Розподіл учасників за групами був випадковим, але через малу вибірку ($n=2$ на групу) не вдалося контролювати баланс за ігровим досвідом.

Зовнішня достовірність. Дослідження проводилося лише з гравцями віком 14-21 років, що не дозволяє екстраполювати результати на старші вікові групи. Тестування на конкретному прототипі 3D-платформера обмежує перенос висновків на інші жанри (бойовики, RPG, головоломки).

Конструктна достовірність. Стан потоку оцінювався через суб'єктивні свідчення респондентів (анкета з шкалою Лайкерта), що є непрямим вимірюванням. Об'єктивні фізіологічні показники (EEG, EDA, eye tracking) не використовувалися. Тому Flow Score свідчить про суб'єктивне сприйняття, а не про реальний психофізіологічний стан.

Статистична достовірність. Обсяг вибірки ($n=8$) не дозволяє застосовувати методи параметричної статистики. Представлені метрики мають характер описової статистики, що свідчить про тенденції, але не про статистично значущі відмінності. Перевірка гіпотез вимагає вибірки ≥ 30 учасників на групу.

4.3 Результати тестування евристичного DDA

Емпіричне тестування евристичного DDA (Група В) підтвердило коректну роботу алгоритму просторової кластеризації на практиці. Зокрема, у контрольному сценарії (учасник В-01) було зафіксовано серію із 2 смертей поспіль на платформі, що обертається (SpinPlatform). Система коректно ідентифікувала кластер: кількість смертей у радіусі 20 м перевищила заданий поріг у 2 подій. Було обчислено координати центроїда, і система відкладено застосувала зміни під час найближчого екранного переходу. Швидкість її обертання та розмір сусідньої платформи були адаптовані на розрахований коефіцієнт 1.20 (полегшення на 20%).

Тест на ускладнення також показав стабільну роботу: коли гравець (учасник В-02) тричі поспіль проходив секцію від першої до другої контрольної точки без жодної смерті, система ініціювала подію OnPlayerDominating. У результаті швидкість MovingPlatform у наступній зоні зросла на 20%. Анкетування показало, що зміна відбулась у момент затемнення екрана і не була

помічена жодним з учасників групи, що задовольнило вимогу невидимості адаптації.

4.4 Результати ML-моделей

Набір даних для навчання ML-моделей був попередньо зібраний під час 14 сесій тестування Baseline-режиму. Загальний обсяг склав 323 марковані приклади ручного зворотного зв'язку від гравців. Розподіл міток цільової змінної: 101 приклад із `desired_multiplier > 1` (запит на полегшення), 199 прикладів із `desired_multiplier < 1` (запит на ускладнення), а 23 приклади, де базовий рівень складності був залишений без змін. Дані було розділено на тренувальну (258 прикладів) та тестову (65 прикладів) вибірки. Модель Random Forest досягла середньоквадратичної похибки $MSE = 0.133$ та коефіцієнта детермінації $R^2 = 0.77$ на тестовій вибірці. Порівняння фактичних оцінок гравців та передбачень моделі Random Forest було наведено раніше (див. рис. 3.11).

Модель багат шарового перцептрону (MLP) показала порівнянну якість: $MSE = 0.150$, $R^2 = 0.734$. Аналіз похибок та залишків для цієї мережі детально розглянуто в попередньому розділі (див. рис. 3.13 та рис. 3.14)

Для наукового обґрунтування доцільності використання розширеного вектора ознак було проведено абляційний аналіз (Ablation Study). Суть цього експерименту полягає у послідовному вилученні окремих метрик із навчальної вибірки та повторному тренуванні моделі Random Forest для ізольованої оцінки їхнього впливу на загальну точність прогнозування. Результати експерименту зведено у таблиці 4.1.

Таблиця 4.1

Результати абляційного аналізу впливу ознак на точність моделі

Конфігурація моделі	Кількість ознак	Коефіцієнт детермінації (R^2)	Відхилення (ΔR^2)
Повний набір (усі метрики)	8	0.765	—
Без тремтіння камери (<code>camera_jitter</code>)	7	0.755	-0.010
Без часу відновлення (<code>time_to_recover</code>)	7	0.757	-0.009
Без часу бездіяльності (<code>idle_time</code>)	7	0.740	-0.025

Продовження Таблиця 4.1

Без частоти вводу (APM)	7	0.739	-0.026
Без обох (jitter, recover)	6	0.752	-0.013
Лише базові (без 4 психологічних метрик)	4	0.596	-0.170

Як свідчать дані таблиці 4.1, послідовне вилучення психологічних індикаторів очікувано знижує якість прогнозування. Критичним є порівняння повного набору ознак із базовим набором (де залишено лише стандартні показники: рівень, кількість смертей тощо). Вилучення чотирьох неявних психологічних метрик спричиняє стрімке падіння коефіцієнта детермінації на 17.0 відсоткових пунктів (із 0.765 до 0.596). Це свідчить про те, що розроблені психологічні метрики є інформативними ознаками, які суттєво підвищують здатність системи точно прогнозувати суб'єктивний стан гравця, і не є просто статистичним шумом.

Важливою метрикою для систем реального часу є час логічного виведення (інференсу) в ігровому рушії. Модель Random Forest, трансльована в чистий C# код за допомогою інструменту m2cgen, продемонструвала середній час виконання 0.04 мс на кадр, що не створює помітного впливу на кадрову частоту (FPS). Модель MLP, що виконувалася через рушій тензорних обчислень Unity Sentis на CPU, показала час виведення (інференсу) 0.8 мс на кадр, що також повністю вписується у бюджет часу для додатків із частотою 60 FPS (16.6 мс на кадр).

Результати моделей з учителем (RF та MLP) показали стабільну якість на статичних даних. Наступним кроком є розгляд результатів агента-Режисера в динамічному середовищі

4.5 Результати RL-тренування

Процес навчання агентів був розділений на дві залежні фази з використанням алгоритму Proximal Policy Optimization (PPO).

Фаза 1 (Тренування BotAgent). Штучний гравець навчався протягом 500 тисяч кроків симуляції з прискоренням часу (TimeScale = 20). Середня сукупна

нагорода (Cumulative Reward) зросла від -0.867 на початку тренування до стабільного плато $+2.5$ на позначці 450 тисяч кроків. На контрольній вибірці епізодів частка успішного завершення рівня ботом досягла 68%. Агент успішно опанував механіку варіативного стрибка. Графіки навчання бота з TensorBoard було наведено в минулому розділі на рис. 3.15.

Фаза 2 (Тренування DirectorAgent). Після замороження ваг штучного гравця розпочалося тренування агента-Режисера (100 тисяч кроків). Оскільки завданням Режисера є фоновий макроменеджмент, традиційна метрика довжини епізоду для нього не є репрезентативною. Натомість якість навчання оцінювалася за фундаментальними метриками функції втрат: Policy Loss та Value Loss (рис. 4.2).

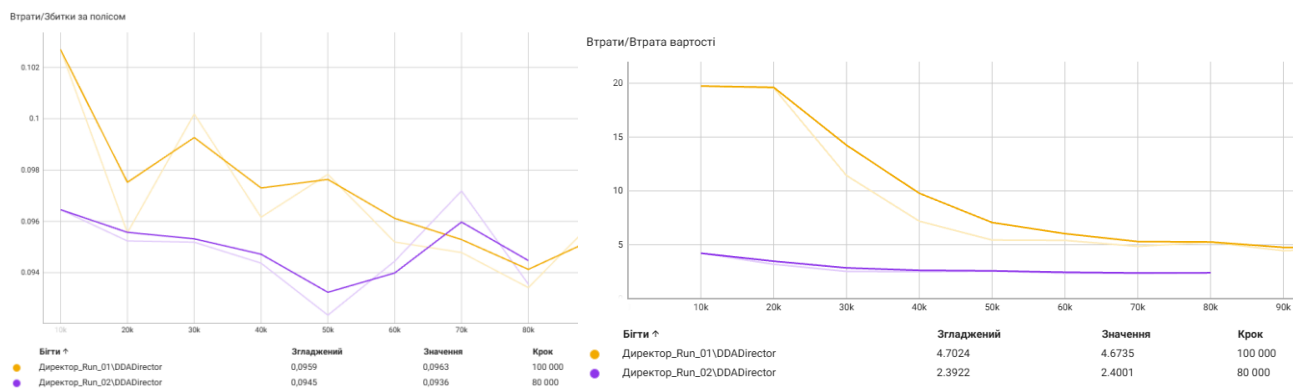


Рис. 4.2 Динаміка навчання DirectorAgent: зміна Policy Loss та Value Loss

Як видно з графіків, показник Value Loss (який відображає похибку критика в оцінці поточного стану залученості гравця) має стійку тенденцію до зниження. Це свідчить про те, що Режисер покращив оцінювання розпізнавати стани фрустрації та нудьги. Водночас конвергенція показника Policy Loss близько нуля вказує на стабілізацію стратегії актора: Режисер навчився своєчасно знижувати множники складності під час тривалих труднощів бота і підвищувати їх у разі надто легкого проходження, підтримуючи метрику успішності (Success Rate) у заданому цільовому діапазоні ≈ 0.80 .

4.6 Порівняльний аналіз підходів

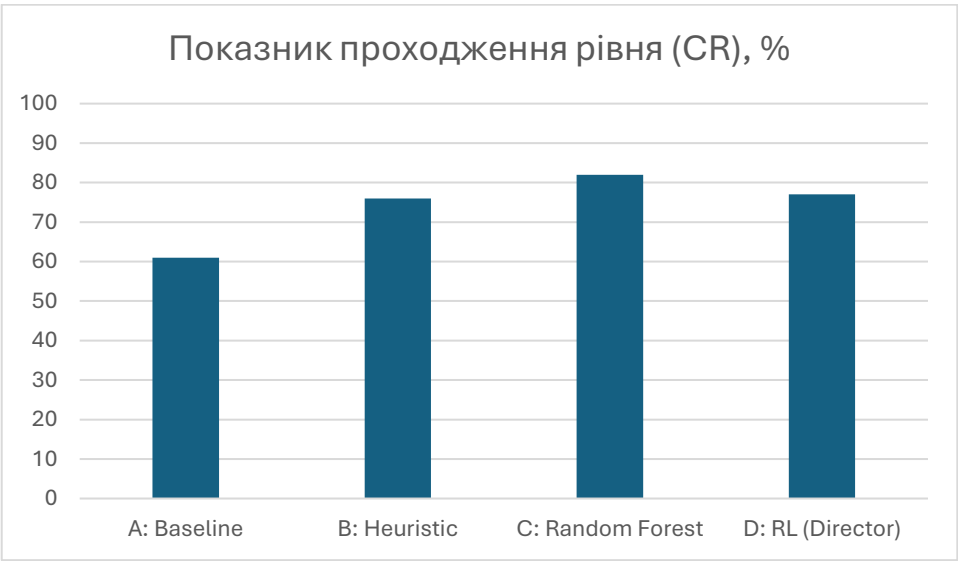
Для проведення об’єктивного порівняльного аналізу та визначення найбільш ефективного рішення, у таблиці 4.2 систематизовано результати фінального А/В-тестування, проведеного серед чотирьох малих фокус-груп. Показники, що використані для оцінки, базуються на системі метрик, детально описаній у підрозділі 4.1, що дозволяє порівняти ефективність кожного з протестованих варіантів.

Таблиця 4.2

Порівняльна оцінка чотирьох варіантів DDA

Група / метрика	CR, %	AD	ST, хв	IS, бали	FS, бали
A: Baseline	61	18.5	18.2	5.0	2.5
B: Heuristic	76	8.5	15.3	3.5	4.1
C: Random Forest	82	10.5	21.5	4.1	4.4
D: RL (Director)	77	11.0	23.8	3.8	4.3

Для більш наочного порівняння динаміки змін та виявлення загальних тенденцій, візуалізацію ключових цільових метрик для всіх чотирьох досліджуваних груп представлено у вигляді стовпчикових діаграм на рис. 4.3.



а)

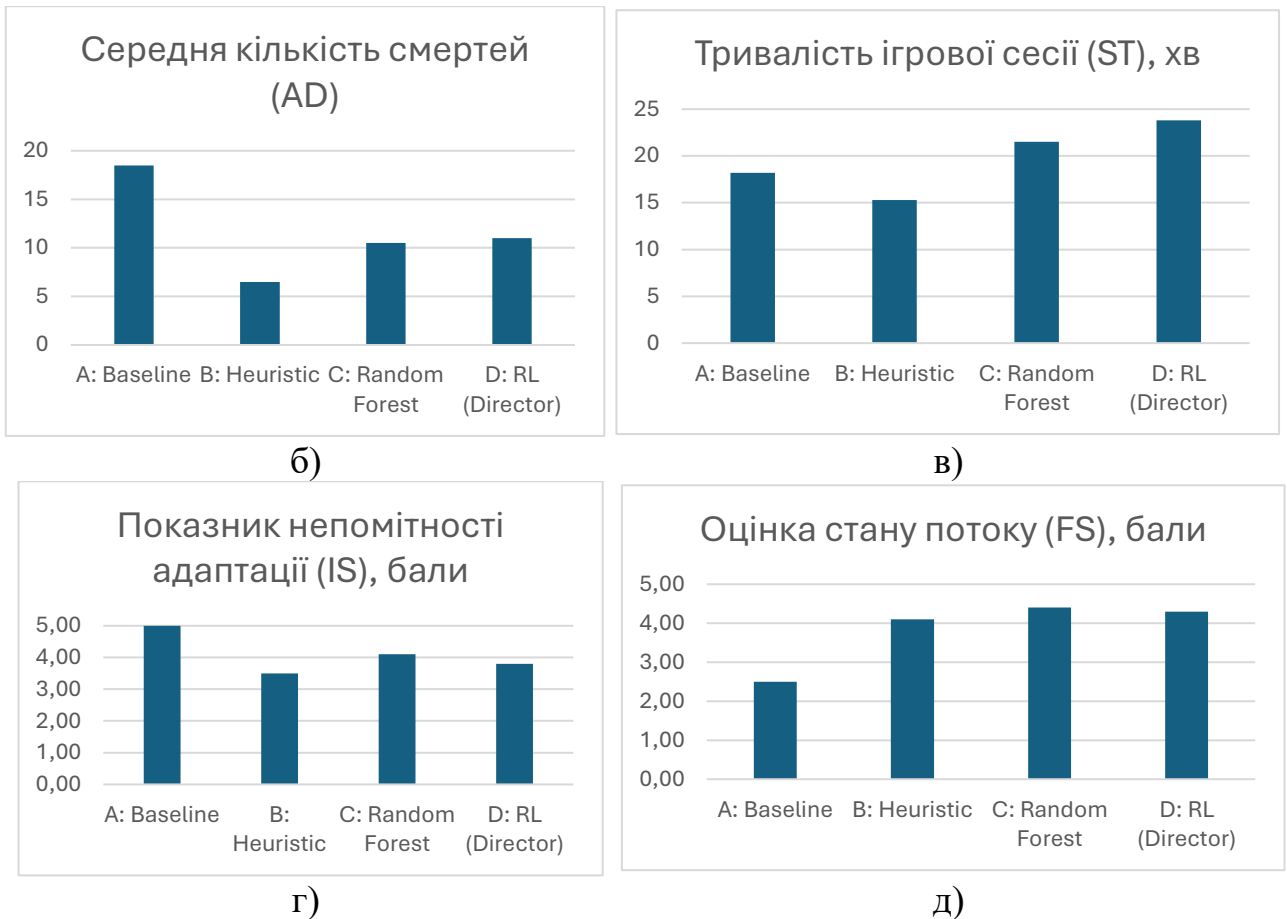


Рис. 4.3 Результати A/B-тестування алгоритмів за ключовими метриками на стовпчикових діаграмах: а) показник проходження рівня (CR); б) середня кількість смертей (AD); в) тривалість ігрової сесії (ST); г) показник непомітності адаптації (IS); д) оцінка стану потоку (FS)

Аналіз даних таблиці 4.2 та діаграм (рис. 4.3) демонструє позитивний вплив систем динамічної адаптації на ігровий досвід. Зафіксовано зростання показника успішного подолання перешкод (Completion Rate) із базових 61% у контрольній групі до 77% у групі, де працював RL-Режисер та досяг максимуму в 82% у групі C (Random Forest). Середня кількість смертей (Average Deaths) суттєво скоротилася: з 18,5 у базовій версії до 10,5–11,0 для ML-моделей та рекордних 8,5 для евристичного алгоритму. Це дозволило ліквідувати головну причину відтоку гравців – фрустрацію від багаторазових невдач.

Показник середнього часу сесії (Session Time) у цьому дослідженні виступає описовою метрикою тривалості проходження. У групах з використанням Random Forest та RL зафіксовано зростання цього показника

порівняно з Baseline. Зокрема, агент на базі RL забезпечив середню тривалість ігрового процесу на рівні 23.8 хв. Водночас у групі з евристичним підходом час сесії скоротився до 15.3 хв (проти 18.2 хв у Baseline), що в поєднанні з найнижчим показником смертності свідчить про швидше проходження рівня після його полегшення, а не про зниження зацікавленості. Варто зазначити, що ці дані відображають динаміку в межах однієї сесії; для оцінки довготривалого утримання гравців (Player Retention) необхідний подальший аналіз повторних сесій протягом тривалого часу [26].

Евристичний DDA (Група В) показав відмінні результати у зниженні смертності, що обґрунтовує його високу цінність як вирішення проблеми "холодного старту" у розробці ігор, де даних для навчання нейронних мереж ще немає. Проте ML-моделі перевершили евристику за показником непомітності змін (Invisibility Score – 4.1 у RF та 3.8 у RL проти 3.5 в евристики). Адаптація виглядала значно «природнішою», оскільки ML-моделі навчилися аналізувати не лише прямі смерті, а й тонкі психологічні індикатори поведінки.

Максимальний рівень залученості (Flow Score) був зафіксований у групі С (4.4 бали), що демонструє перспективність класичних алгоритмів на кшталт Random Forest. Агент-режисер на базі навчання з підкріпленням (Flow Score = 4.3) продемонстрував результати, схожі з лідером. Це підтверджує його придатність як інструмента для часткової автоматизації завдань ігрового балансування, хоча в поточній ітерації простору станів він не показав максимальних значень одночасно за всіма метриками. [27, 30].

Висновки до розділу 4

У четвертому розділі описано методологію проведення незалежного А/В-тестування DDA-систем із залученням фокус-груп. Результати емпіричних досліджень вказують на тенденцію до покращення ключових цільових метрик за умови використання методів динамічної адаптації порівняно зі статичною складністю. Оцінка якості моделей (зокрема Random Forest та MLP) на тестовій вибірці (за метриками MSE та R2) підтвердила їхню здатність адекватно

прогнозувати оптимальний рівень складності на основі поведінкової телеметрії. Водночас результати А/В-тестування засвідчили, що серед інтегрованих алгоритмів саме Random Forest забезпечив найвищий показник проходження рівня (82 %) та найкращий стан потоку (4,4 з 5,0) серед досліджуваних підходів.

Аналіз метрик Policy Loss та Value Loss підтвердив збіжність цільової функції агента-Режисера, що підтримує гіпотезу про життєздатність запропонованої двоагентної архітектури, натхненної концепцією Asymmetric Self-Play для розробки 3D-платформерів. І хоча RL-модель не здобула абсолютної першості за кожною окремою метрикою, вона відзначилася стабільністю, забезпечивши найдовшу тривалість ігрової сесії з-поміж використаних інтелектуальних систем.

Важливим практичним результатом є доведення того, що навіть базова евристична DDA-система з математичною моделлю просторової кластеризації дає значний приріст частки успішно подоланих перешкод (+15 відсоткових пунктів) і найбільш радикально знижує показник фрустрації (зменшення середньої кількості смертей майже втричі). Це свідчить на користь наукової та практичної цінності розробленого програмного комплексу, закладених у нього архітектурних рішень та обраного вектора розвитку гібридних DDA-систем.

ВИСНОВКИ

У бакалаврській кваліфікаційній роботі досліджено та практично реалізовано систему динамічної адаптації складності (DDA) рівнів для 3D-платформера, побудованого на базі рушія Unity. Розроблена система охоплює повний технічний цикл: від збору поведінкових метрик гравця до навчання і розгортання адаптивних алгоритмів безпосередньо в ігровому середовищі виконання.

У результаті виконання роботи отримано такі результати:

1. Проведено аналіз предметної області DDA. Розглянуто комерційні реалізації систем складності (у проєктах Resident Evil 4, Left 4 Dead, Crash Bandicoot) і встановлено їхнє спільне обмеження – відсутність просторового та психологічного виміру у вхідних метриках. Виявлено, що більшість сучасних академічних робіт із ML-DDA зосереджені на 2D-іграх із спрощеним простором станів, що підкреслює наукову новизну та актуальність проведеного дослідження саме для 3D-платформерів. Розроблена система ширший спектр метрик порівняно з розглянутими комерційними рішеннями за повнотою використаних метрик і здатністю навчатися на реальних даних.

2. Розроблено підсистему неперервного збору телеметрії. Визначено та формалізовано вектор із восьми поведінкових метрик (рівень, смерті на контрольній точці, кластеризація смертей, частоти вводу, час на рівні, час на відновлення, час бездіяльності, тремтіння камери), що комплексно охоплюють просторовий і психологічний стани гравця. Архітектура модулів GameSession та AnalyticsManager забезпечує асинхронний запис журналу подій у форматі NDJSON без негативного впливу на кадрову частоту (FPS) [29].

3. Реалізовано евристичну DDA-систему на основі просторової кластеризації смертей із урахуванням останньої безпечної позиції гравця (Last Safe Position). Під час тестування показник успішного подолання перешкод (Completion Rate) зріс від 61% до 76% порівняно з базовою версією гри, а середня кількість смертей зменшилася з 18.5 до 8.5. Механізм буферизації змін та їхнього

застосування під час візуального екранного переходу забезпечив показник непомітності адаптації на рівні 3.5 з 5.0 балів.

4. Побудовано наскрізний ML-конвеєр. На базі 323 навчальних прикладів, зібраних за принципом «людина-в-контурі» (Human-in-the-Loop), навчено регресійні моделі Random Forest ($R^2 = 0.77$) та багатошаровий перцептрон MLP ($R^2 = 0.73$). Модель Random Forest транслювалась у чистий C#-код через інструмент m2cgen, а MLP – у формат ONNX для рушія Unity Sentis. За результатами А/В-тестування Random Forest забезпечив найвищий показник успішного подолання перешкод (Completion Rate = 82%) серед досліджуваних підходів та найкращу суб'єктивну оцінку залученості (Flow Score = 4.4).

5. Спроектовано і реалізовано систему на базі навчання з підкріпленням (RL) із двома агентами в межах архітектури, натхненної підходами Asymmetric Self-Play. Штучний гравець (BotAgent) досяг частки успішного проходження рівня у 68% за 500 тисяч імітаційних кроків. Агент-режисер (DirectorAgent), використовуючи замороженого бота як симулятора, зміг утримувати його у цільовій зоні "потoku" (діапазон успішності 0.79–0.82). Застосування RL-підходу забезпечило результати на рівні CR = 77% і Flow Score = 4.3, а також згенерувало найдовші за тривалістю ігрові сесії (23.8 хв), що вказує на потенціал цієї архітектури для часткової автоматизації балансування складності.

6. Проведено комплексне А/В-тестування чотирьох розроблених варіантів DDA на малій фокус-групі з 8 учасників. Хоча обмежений обсяг вибірки не дозволяє робити висновки про високу статистичну значущість, результати виявили позитивні тенденції та попередньо підтверджують доцільність впроваджених алгоритмічних покращень за визначеними цільовими метриками.

Практичне значення роботи полягає у тому, що створена система є відкритим, гнучким і відтворюваним програмним прототипом. Підтверджено придатність архітектурних рішень – зокрема імплементація патерну «Стратегія», механізму буферизації через подію OnScreenDarkened та алгоритму захисту від відкоту складності ddaProgressTracker, мають високий ступінь готовності та можуть бути безпосередньо перенесені у комерційні проєкти ігрових студій.

Отримані аналітичні результати показують практичний результат, і можуть слугувати основою для подальших досліджень розуміння порівняльної ефективності різних підходів до динамічної адаптації у 3D-середовищах.

Для забезпечення доступності результатів дослідження вихідний код із супровідною документацією опубліковано у відкритому репозиторії [37], а демонстраційну збірку гри розміщено на платформі itch.io [38]. Це дозволяє незалежно відтворити, перевірити або розширити отримані результати без необхідності звернення до авторів.

Поряд з досягнутими результатами автор усвідомлює окремі недопрацювання роботи. Не вдалося провести великомасштабне тестування (≥ 30 учасників) у зв'язку з обмеженими часовими ресурсами. Не вдалося порівняти PPO з SAC або A2C через тривалість одного циклу тренування. Цілісне дослідження довготривалого ефекту DDA на показник утримання гравців (Player Retention) потребує окремого продовжуваного проєкту тривалістю 6-12 місяців.

Перспективним напрямком подальшого розвитку є навчання DirectorAgent безпосередньо на живих сесіях із реальними гравцями замість використання ботів симуляторів, застосування рекурентних нейронних мереж (RNN) для аналізу часових рядів поведінки, а також розширення простору дій Режисера інструментами для глобального управління подіями, змістовною структурою та архітектурою (топологією) самого рівня. Окремим напрямком є адаптація розробленого дослідницького середовища для використання у навчальному процесі кафедри в рамках курсів з машинного навчання та розробки ігрового програмного забезпечення.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Global Games Market Report [Електронний ресурс] / Newzoo. Amsterdam, 2025. 67 p. URL: https://investgame.net/wp-content/uploads/2025/09/2025_Newzoo_Free_Global_Games_Market_Report.pdf (дата звернення: 28.04.2026)
2. Zohaib M. Dynamic Difficulty Adjustment (DDA) in Computer Games: A Review / M. Zohaib // Advances in Human-Computer Interaction. 2018. Vol. 2018. P. 1-12.
3. Csikszentmihalyi M. Flow: The Psychology of Optimal Experience. New York: Harper & Row, 1990. 303 p.
4. Hunicke R. The Case for Dynamic Difficulty Adjustment in Games / R. Hunicke // Proceedings of the 2005 ACM SIGCHI International Conference on Advances in Computer Entertainment Technology. New York, 2005. P. 429-433.
5. Charles D. Dynamic Player Modelling: A Framework for Player-Centered Digital Games / D. Charles, M. Black // Proceedings of the International Conference on Computer Games: AI, Design and Education. Reading, 2004. P. 29-35.
6. Hunicke R. AI for Dynamic Difficulty Adjustment in Games. AAAI Workshop on Challenges in Game AI/ R. Hunicke, Chapman V.// AAAI Workshop on Challenges in Game AI. 2004. P. 1-4.
7. Campbell J. Automated difficulty adjustment in video games: A machine learning approach / J. Campbell, W. Ku // Proceedings of the International Conference on Machine Learning and Applications. 2020. P. 1-6.
8. Sutton R. S. Reinforcement Learning: An Introduction / R. S. Sutton, A. G. Barto. 2nd ed. Cambridge: MIT Press, 2018. 526 p.
9. Silver D. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play / D. Silver et al. // Science. 2018. Vol. 362(6419). P. 1140-1144.
10. Nacke L. Towards a Framework of Player Experience Research / L. Nacke, A. Drachen // Proceedings of the Second International Workshop on Evaluating Player Experience in Games at FDG. 2011.

11. Breiman L. Random Forests / L. Breiman // Machine Learning. 2001. Vol. 45(1). P. 5-32.
12. Mnih V. Human-Level Control through Deep Reinforcement Learning / V. Mnih et al. // Nature. 2015. Vol. 518. P. 529-533.
13. LeCun Y. Deep Learning / Y. LeCun, Y. Bengio, G. Hinton // Nature. 2015. Vol. 521. P. 436-444.
14. Bansal T. Emergent Complexity via Multi-Agent Competition / T. Bansal et al. // arXiv preprint arXiv:1710.03748. 2017.
15. Pedregosa F. Scikit-learn: Machine Learning in Python / F. Pedregosa et al. // Journal of Machine Learning Research. 2011. Vol. 12. P. 2825-2830.
16. Juliani A. Unity: A General Platform for Intelligent Agents / A. Juliani et al. // arXiv preprint arXiv:1809.01500. 2020.
17. Schulman J. Proximal Policy Optimization Algorithms / J. Schulman et al. // arXiv preprint arXiv:1707.06347. 2017.
18. Isaza C. m2cgen: Transform your ML model into a native code [Електронний ресурс] / C. Isaza. URL: <https://github.com/BayesWitnesses/m2cgen> (дата звернення: 16.04.2026).
19. Документація Unity Engine [Електронний ресурс]. URL: <https://docs.unity3d.com/Manual/index.html> (дата звернення: 02.08.2025).
20. Документація Unity ML-Agents Toolkit [Електронний ресурс]. URL: <https://github.com/Unity-Technologies/ml-agents/tree/main/docs> (дата звернення: 28.05.2026).
21. Документація Unity Sentis [Електронний ресурс]. URL: <https://docs.unity3d.com/Packages/com.unity.sentis@latest> (дата звернення: 14.11.2025).
22. Kolen J. Dynamic Difficulty Adjustment for Maximized Engagement in Digital Games / J. Kolen et al. // IEEE Computational Intelligence and Games. 2015. P. 470-477.
23. Skabar A. Game Difficulty Adaptation: A Survey of Methods, Applications and Studies / A. Skabar, K. Amirghasemi // IEEE Transactions on Games. 2020. Vol. 12(2). P. 103-117.

24. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall, 2017. 432 p.
25. Gamma E. Design Patterns: Elements of Reusable Object-Oriented Software / E. Gamma et al. Boston: Addison-Wesley, 1994. 395 p.
26. Víteková L. Dynamic Difficulty Adjustment in Serious Games: A Literature Review [Электронный ресурс] / L. Víteková, C. Eichhorn, J. Pirker, D. A. Plecher // Information. 2026. Vol. 17, No. 1. Art. 96. DOI: 10.3390/info17010096. URL: <https://www.mdpi.com/2078-2489/17/1/96> (дата звернення: 05.04.2026)
27. Fisher N. Dynamic Difficulty Adjustment in Games: Concepts, Techniques, and Applications [Электронный ресурс] / N. Fisher, A. K. Kulshreshth // From Pixels to Play — The Art and Science of Video Games / ed. T. Guzsvinecz. London: IntechOpen, 2025. DOI: 10.5772/intechopen.1011703. URL: <https://www.intechopen.com/chapters/1228576#> (дата звернення: 21.03.2026)
28. Burke A. Using Player Profiling to Enhance Dynamic Difficulty Adjustment in Video Games: Senior Project [Электронный ресурс] / A. Burke. California Polytechnic State University, San Luis Obispo, 2012. URL: <https://digitalcommons.calpoly.edu/cpesp/76> (дата звернення: 28.11.2025).
29. Rahimi M. Continuous Reinforcement Learning-based Dynamic Difficulty Adjustment in a Visual Working Memory Game [Электронный ресурс] / M. Rahimi, H. Moradi, A. Vahabie, H. Kebriaei // arXiv preprint arXiv:2308.12726. 2023. URL: <https://arxiv.org/abs/2308.12726> (дата звернення: 19.03.2026).
30. Dynamic Difficulty Adjustment in Games using Reinforcement Learning: Master's Thesis [Электронный ресурс] / Purdue University. West Lafayette, 2024. Purdue HAMMER Repository. DOI: 10.25394/PGS.30790583. URL: https://hammer.purdue.edu/articles/thesis/Dynamic_Difficulty_Adjustment_in_Games_using_Reinforcement_Learning/30790583?file=60330848 (дата звернення: 15.12.2025)
31. GameAnalytics. Mobile Gaming Benchmarks 2026: Industry Report [Электронный ресурс]. URL: <https://www.gameanalytics.com/reports/2026-mobile-pc-gaming-benchmarks> (дата звернення: 14.05.2026).
32. Mighty No. 9 critic reviews [Электронный ресурс] // Metacritic. URL:

<https://www.metacritic.com/game/mighty-no-9/critic-reviews> (дата звернення: 05.05.2026).

33. Narcisse E. Crash Bandicoot Remaster Trilogy Gets Long Lost Level As DLC [Електронний ресурс] / E. Narcisse. 2017. URL: <https://kotaku.com/crash-bandicoot-remaster-trilogy-gets-long-lost-level-a-1797104488> (дата звернення: 15.05.2026).

34. Aponte M. A. Difficulty in Video Games: Definitions, Design Principles, and the Player's Role / M. A. Aponte, G. Levieux, S. Natkin // Proceedings of the 10th International Conference on Entertainment Computing. 2011. P. 134-143.

35. Sonic the Hedgehog (2006) critic reviews [Електронний ресурс] // Metacritic. URL: <https://www.metacritic.com/game/sonic-the-hedgehog-2006/critic-reviews> (дата звернення: 20.05.2026).

36. Cuphead: The Delicious Last Course reaches 2 million copies sold [Електронний ресурс] // Studio MDHR official communications, 2022. URL: <https://www.gamedeveloper.com/business/-i-cuphead-the-delicious-course-i-reaches-2-million-copies-sold> (дата звернення: 20.05.2026).

37. Верещак Б. О. Intruder – вихідний код системи динамічної адаптації складності [Електронний ресурс] / Б. О. Верещак. 2026. URL: https://github.com/BogdanVereshchak/Intruder_Public (дата звернення: 02.06.2026)

38. Верещак Б. О. Intruder – демонстраційна збірка гри [Електронний ресурс] / Б. О. Верещак. 2026. URL: <https://luxpro.itch.io/intruder> (дата звернення: 02.06.2026)

ДОДАТОК А

Перелік ключових компонентів системи

A.1 Інтерфейс IDDAStrategy

```
namespace DDA.Strategies
{
    /// <summary>
    /// Base interface for Dynamic Difficulty Adjustment algorithms.
    /// Ensures Open/Closed Principle (OCP): new DDA types can be added without
    changing the DDAManager.
    /// </summary>
    public interface IDDAStrategy
    {
        void HandlePlayerStuck(Vector3 troublePosition,
HashSet<IDynamicObstacle> obstacles);
        void HandlePlayerCruising(Vector3 playerPosition,
HashSet<IDynamicObstacle> obstacles);
    }
}
```

A.2 Фрагмент HeuristicDDAStrategy

```
public class HeuristicDDAStrategy : IDDAStrategy
{
    private float adaptationRadius;
    private float complicationRadius;
    private float easingFactor;
    private float complicationFactor;

    public HeuristicDDAStrategy(float adaptRadius, float compRadius, float
ease, float complicate)
    {
        this.adaptationRadius = adaptRadius;
        this.complicationRadius = compRadius;
        this.easingFactor = ease;
        this.complicationFactor = complicate;
    }

    public void HandlePlayerStuck(Vector3 troublePosition,
HashSet<IDynamicObstacle> obstacles)
    {
        Debug.Log($"[Heuristic DDA] Easing obstacles near:
{troublePosition}");
        GameEvents.OnGlobalTensionChanged?.Invoke(easingFactor);
        foreach (var obstacle in obstacles)
        {
            if (obstacle is MonoBehaviour mb)
            {
                float distance = Vector3.Distance(mb.transform.position,
troublePosition);
                if (distance <= adaptationRadius)
                {
                    float distanceWeight = 1f - (distance /
adaptationRadius);
                    float effectiveMultiplier = Mathf.Lerp(1f, easingFactor,
distanceWeight);
                    obstacle.AdjustDifficulty(effectiveMultiplier, mode:
AdjustmentMode.ForcedOverride);
                }
            }
        }
    }
}
```

```

    }
    }
}

public void HandlePlayerCruising(Vector3 playerPosition,
HashSet<IDynamicObstacle> obstacles)
{
    Debug.Log("[Heuristic DDA] Player is dominating! Complicating
upcoming obstacles.");
    GameEvents.OnGlobalTensionChanged?.Invoke(complicationFactor);
    foreach (var obstacle in obstacles)
    {
        if (obstacle is MonoBehaviour mb)
        {
            float distance = Vector3.Distance(mb.transform.position,
playerPosition);
            if (distance <= complicationRadius)
            {
                float distanceWeight = 1f - (distance /
complicationRadius);
                float effectiveMultiplier = Mathf.Lerp(1f,
complicationFactor, distanceWeight);
                obstacle.AdjustDifficulty(effectiveMultiplier, mode:
AdjustmentMode.ForcedOverride);
            }
        }
    }
}

```

A.3 Фрагмент y DinamicObstacleBase

```

public void AdjustDifficulty(float multiplier, AdjustmentMode mode =
AdjustmentMode.Buffered)
{
    if (Mathf.Approximately(ddaWeight, 0f)) return;

    float weightedMultiplier = Mathf.Lerp(1f, multiplier, ddaWeight);
    if (mode != AdjustmentMode.ForcedOverride && weightedMultiplier >= 1f &&
weightedMultiplier < ddaProgressTracker)
        return;

    ddaProgressTracker = weightedMultiplier;
    pendingDifficultyRatio = weightedMultiplier;
    hasPendingChanges = true;

    if (mode == AdjustmentMode.Immediate || mode ==
AdjustmentMode.ForcedOverride)
        ApplyPendingChanges();
}

private void HandleScreenDarkened()
{
    ApplyPendingChanges();
}

private void ApplyPendingChanges()
{
    if (!hasPendingChanges) return;

    // Застосовуємо зміни ФІЗИЧНО
    ApplyDDA(pendingDifficultyRatio);
    hasPendingChanges = false;
}

```

```

    }
    protected abstract void ApplyDDA(float multiplier);

```

A.4 Скрипт MachineLearningDDAStrategy

```

public class MachineLearningDDAStrategy : IDDAStrategy
{
    private DDAInferenceManager inferenceManager;
    private float adaptationRadius;

    public MachineLearningDDAStrategy(DDAInferenceManager inferenceManager,
float adaptationRadius)
    {
        this.inferenceManager = inferenceManager;
        this.adaptationRadius = adaptationRadius;
    }

    public void HandlePlayerStuck(Vector3 troublePosition,
HashSet<IDynamicObstacle> obstacles)
    {
        if (inferenceManager == null) return;

        float level = GameSession.Instance != null ?
GameSession.Instance.CurrentLevel : 1f;
        float checkpointDeaths = GameSession.Instance != null ?
GameSession.Instance.CheckpointDeaths : 0f;
        float clusterDeaths = GameSession.Instance != null ?
GameSession.Instance.ClusterDeaths : 0f;
        float inputRate = GameSession.Instance != null ?
GameSession.Instance.CurrentInputRate : 0f;
        float timeSpent = GameSession.Instance != null ?
GameSession.Instance.TimeSpentOnLevel : 0f;
        float recoveryTime = GameSession.Instance != null ?
GameSession.Instance.AverageTimeToRecover : 0f;
        float idleTime = GameSession.Instance != null ?
GameSession.Instance.TotalIdleTimeThisLevel : 0f;
        float cameraJitter = GameSession.Instance != null ?
GameSession.Instance.TotalCameraJitter : 0f;

        float predictedMultiplier =
inferenceManager.PredictDesiredMultiplier(
            level,
            checkpointDeaths,
            clusterDeaths,
            inputRate,
            timeSpent,
            recoveryTime,
            idleTime,
            cameraJitter
        );

        GameEvents.OnGlobalTensionChanged?.Invoke(predictedMultiplier);
        foreach (var obstacle in obstacles)
        {
            if (obstacle is MonoBehaviour mb)
            {
                float distance = Vector3.Distance(mb.transform.position,
troublePosition);
                if (distance <= adaptationRadius)
                {
                    float distanceWeight = 1f - (distance /
adaptationRadius);

```

```

        float effectiveMultiplier = Mathf.Lerp(1f,
predictedMultiplier, distanceWeight);
        obstacle.AdjustDifficulty(effectiveMultiplier,
AdjustmentMode.Buffered);
    }
}

    public void HandlePlayerCruising(Vector3 playerPosition,
HashSet<IDynamicObstacle> obstacles)
    {
        if (inferenceManager == null) return;
        float level = GameSession.Instance != null ?
GameSession.Instance.CurrentLevel : 1f;
        float inputRate = GameSession.Instance != null ?
GameSession.Instance.CurrentInputRate : 0f;
        float timeSpent = GameSession.Instance != null ?
GameSession.Instance.TimeSpentOnLevel : 0f;
        float recoveryTime = GameSession.Instance != null ?
GameSession.Instance.AverageTimeToRecover : 0f;
        float idleTime = GameSession.Instance != null ?
GameSession.Instance.TotalIdleTimeThisLevel : 0f;
        float cameraJitter = GameSession.Instance != null ?
GameSession.Instance.TotalCameraJitter : 0f;
        float predictedMultiplier =
inferenceManager.PredictDesiredMultiplier(level, 0, 0, inputRate, timeSpent,
recoveryTime, idleTime, cameraJitter);
        GameEvents.OnGlobalTensionChanged?.Invoke(predictedMultiplier);

        foreach (var obstacle in obstacles)
        {
            if (obstacle is MonoBehaviour mb)
            {
                float distance = Vector3.Distance(mb.transform.position,
playerPosition);
                if (distance <= adaptationRadius)
                {
                    float distanceWeight = 1f - (distance /
adaptationRadius);
                    float effectiveMultiplier = Mathf.Lerp(1f,
predictedMultiplier, distanceWeight);
                    obstacle.AdjustDifficulty(effectiveMultiplier, mode:
AdjustmentMode.ForcedOverride);
                }
            }
        }
    }

    public void HandlePlayerDeath(Vector3 deathPosition,
HashSet<IDynamicObstacle> obstacles)
    {
        HandlePlayerStuck(deathPosition, obstacles);
    }

    public void HandleCheckpointReached(Vector3 checkpointPosition,
HashSet<IDynamicObstacle> obstacles)
    {
        HandlePlayerCruising(checkpointPosition, obstacles);
    }
}

```

A.5 Скрипт ReinforcementLearningDDAStrategy

```
namespace DDA.Strategies
{
    public class ReinforcementLearningDDAStrategy : IDDAStrategy
    {
        private DDADirectorAgent directorAgent;

        public ReinforcementLearningDDAStrategy(DDADirectorAgent agent)
        {
            this.directorAgent = agent;
        }

        public void HandlePlayerStuck(Vector3 troublePosition,
        HashSet<IDynamicObstacle> obstacles)
        {
            directorAgent?.RequestDecision();
        }

        public void HandlePlayerCruising(Vector3 playerPosition,
        HashSet<IDynamicObstacle> obstacles)
        {
            directorAgent?.RequestDecision();
        }

        public void HandlePlayerDeath(Vector3 deathPosition,
        HashSet<IDynamicObstacle> obstacles)
        {
            directorAgent?.NotifyBotDied();
        }

        public void HandleCheckpointReached(Vector3 checkpointPosition,
        HashSet<IDynamicObstacle> obstacles)
        {
            directorAgent?.NotifyCheckpointReached();
        }
    }
}
```

A.6 Фрагмент BotAgent

```
[RequireComponent(typeof(PlayerMovement))]  
public class BotAgent : Agent  
{  
    [Header("Dependencies")]  
    [SerializeField] RespawnSystem      respawnSystem;  
    [SerializeField] BotCognitiveState  cognitiveState;  
    [SerializeField] LevelGraph         levelGraph;  
    [SerializeField] Transform          nextCheckpointTransform; // updated by  
    RLEnvController  
  
    [Header("Normalisation caps")]  
    [SerializeField] float maxSpeed      = 12f;  
    [SerializeField] float maxDist      = 60f;  
    [SerializeField] float maxHeightDelta = 25f;  
    [SerializeField] float maxTimeSinceDeath = 30f;  
  
    [Header("Rewards")]  
    [SerializeField] float rewardCheckpoint = 1.0f;  
    [SerializeField] float rewardDeath     = -1.0f;  
    [SerializeField] float rewardObstaclePass = 0.20f;  
    [SerializeField] float rewardProgressScale = 0.005f;  
}
```

```

[SerializeField] float penaltyPerStep      = -0.001f;

private PlayerMovement movement;
private Rigidbody      rb;
private Health         health;
private float          jumpHoldTime;
private float          timeSinceLastDeath;
private int            episodeSteps;
private bool           jumpHeld;

public override void CollectObservations(VectorSensor sensor)
{
    // [0-2] velocity
    Vector3 vel = rb != null ? rb.linearVelocity : Vector3.zero;
    sensor.AddObservation(vel.x / maxSpeed);
    sensor.AddObservation(vel.y / maxSpeed);
    sensor.AddObservation(vel.z / maxSpeed);

    // [3] grounded
    sensor.AddObservation(movement.IsGrounded() ? 1f : 0f);

    // [4] jump hold time
    sensor.AddObservation(Mathf.Clamp01(jumpHoldTime / 0.5f));

    // [5] vertical velocity
    sensor.AddObservation(Mathf.Clamp(vel.y / maxSpeed, -1f, 1f));

    // [6] cognitive load
    sensor.AddObservation(cognitiveState != null ?
cognitiveState.CognitiveLoad : 0f);

    // [7] time since last death
    sensor.AddObservation(Mathf.Clamp01(timeSinceLastDeath /
maxTimeSinceDeath));

    // [8-12] navigation
    if (nextCheckpointTransform != null)
    {
        Vector3 delta = nextCheckpointTransform.position -
transform.position;
        float dist = delta.magnitude;
        Vector3 dir = dist > 0.01f ? delta.normalized : Vector3.forward;
        sensor.AddObservation(dir.x);
        sensor.AddObservation(dir.y);
        sensor.AddObservation(dir.z);
        sensor.AddObservation(Mathf.Clamp01(dist / maxDist));
        sensor.AddObservation(Mathf.Clamp(delta.y / maxHeightDelta, -1f,
1f));
    }
    else
    {
        sensor.AddObservation(0f); sensor.AddObservation(0f);
sensor.AddObservation(0f);
        sensor.AddObservation(0f); sensor.AddObservation(0f);
    }

    // [13] episode progress
    int maxStep = MaxStep > 0 ? MaxStep : 7000;
    sensor.AddObservation((float)episodeSteps / maxStep);
}

public override void OnActionReceived(ActionBuffers actions)
{
    episodeSteps++;
}

```

```

timeSinceLastDeath += Time.fixedDeltaTime;

int moveAction = actions.DiscreteActions[0];
bool holdJump = actions.DiscreteActions[1] == 1;

Vector2 moveInput = moveAction switch
{
    1 => Vector2.up,
    2 => Vector2.down,
    3 => Vector2.left,
    4 => Vector2.right,
    _ => Vector2.zero,
};

movement.SetBotInput(moveInput, holdJump);

if (holdJump) jumpHoldTime += Time.fixedDeltaTime;
else jumpHoldTime = 0f;
jumpHeld = holdJump;

if (nextCheckpointTransform != null && rb != null)
{
    Vector3 dir = (nextCheckpointTransform.position -
transform.position).normalized;
    float dot = Vector3.Dot(rb.linearVelocity.normalized, dir);
    float prog = Mathf.Max(0f, dot);
    AddReward(rewardProgressScale * prog);
}

AddReward(penaltyPerStep);
}

```

A.7 Фрагмент DDADirectorActor

```

public void NotifyBotDied()
{
    deathsSinceLastCheckpoint++;

    if (deathsSinceLastCheckpoint > 4)
    {
        AddReward(-0.1f);
    }
    RequestDecision();
}

public void NotifyCheckpointReached()
{
    if (localGraph == null) FindGraphManager();
    float timeSpent = Time.time - timeAtLastCheckpoint;

    // =====
    // ФОРМУЛА ПОТОКУ (FLOW ZONE REWARD)
    // =====
    if (deathsSinceLastCheckpoint == 0)
    {
        AddReward(-0.2f);
    }
    else if (deathsSinceLastCheckpoint >= 1 && deathsSinceLastCheckpoint
<= 3)
    {
        // ИДЕАЛЬНО! Бот трохи постраждав, але подолав перешкоду.
        AddReward(1.0f);
    }
}

```

```

else
{
    // НАДТО СКЛАДНО! Бот помирав більше 3 разів. Фрустрація.
    AddReward(-1.0f);
}

// Запитуємо нове рішення (Змінюємо складність для наступної зони)
RequestDecision();

// Скидаємо лічильники для наступної зони
deathsSinceLastCheckpoint = 0;
timeAtLastCheckpoint = Time.time;
}

public override void CollectObservations(VectorSensor sensor)
{
    if (localGraph == null) FindGraphManager();
    // 1. Статистика (Локальна)
    sensor.AddObservation(deathsSinceLastCheckpoint);
    sensor.AddObservation(Time.time - timeAtLastCheckpoint);

    // 2. Стан вікна перешкод (Читали з LevelGraphManager)
    if (localGraph != null)
    {
        var window = localGraph.GetSlidingWindow(lookBehind, lookAhead);
        for (int i = 0; i < window.Length; i++)
        {
            sensor.AddObservation(window[i] != null ?
window[i].CurrentMultiplier : 0f);
        }
    }
    else
    {
        for (int i = 0; i < windowSize; i++) sensor.AddObservation(0f);
    }
}

public override void OnActionReceived(ActionBuffers actions)
{
    if (localGraph == null) FindGraphManager();
    var continuousActions = actions.ContinuousActions;

    if (localGraph != null)
    {
        Debug.Log($"DDADirectorAgent applying actions: {string.Join(",
", continuousActions)} to window around bot.");
        var window = localGraph.GetSlidingWindow(lookBehind, lookAhead);
        for (int i = 0; i < window.Length; i++)
        {
            if (window[i] != null && i < continuousActions.Length)
            {
                float obstacleMultiplier =
MapActionToMultiplier(continuousActions[i]);
                window[i].AdjustDifficulty(obstacleMultiplier,
DDA.Interfaces.AdjustmentMode.ForcedOverride);
            }
        }
    }
}

```

A.8 Скрипт DDAIInferenceManager

```

public class DDAIInferenceManager : MonoBehaviour
{

```

```

public enum ModelType { NeuralNetwork_ONNX, RandomForest_Native }

[Header("Model Selection")]
[Tooltip("Обери, який алгоритм використовувати для розрахунку DDA")]
public ModelType currentModel = ModelType.RandomForest_Native;

[Header("Sentis Setup (Only for Neural Net)")]
public ModelAsset ddaModelAsset;

[Tooltip("JSON файл з параметрами нормалізації (mean та scale)")]
public TextAsset scalerJsonAsset;

private Worker m_Worker;

private float[] scalerMeans = new float[8];
private float[] scalerScales = new float[8];

private double[] cachedDoubleArgs = new double[8];
private float[] cachedFloatArgs = new float[8];
private TensorShape cachedTensorShape;
private bool isInitialized = false;

public float PredictDesiredMultiplier(
    float level, float checkpointDeaths, float clusterDeaths, float
inputRate,
    float timeSpent, float timeToRecover, float idleTime, float jitter)
{
    if (currentModel == ModelType.RandomForest_Native)
    {
        cachedDoubleArgs[0] = level;
        cachedDoubleArgs[1] = checkpointDeaths;
        cachedDoubleArgs[2] = clusterDeaths;
        cachedDoubleArgs[3] = inputRate;
        cachedDoubleArgs[4] = timeSpent;
        cachedDoubleArgs[5] = timeToRecover;
        cachedDoubleArgs[6] = idleTime;
        cachedDoubleArgs[7] = jitter;

        return (float)RandomForestModel.Score(cachedDoubleArgs);
    }
    else
    {
        if (m_Worker == null) return 1f;

        float[] rawInputs = { level, checkpointDeaths, clusterDeaths,
inputRate, timeSpent, timeToRecover, idleTime, jitter };

        for (int i = 0; i < 8; i++)
        {
            float safeScale = (scalerScales[i] == 0f) ? 1f :
scalerScales[i];
            cachedFloatArgs[i] = (rawInputs[i] - scalerMeans[i]) /
safeScale;
        }
        using Tensor<float> inputTensor = new
Tensor<float>(cachedTensorShape, cachedFloatArgs);
        m_Worker.Schedule(inputTensor);
        var outputTensor = m_Worker.PeekOutput() as Tensor<float>;
        return outputTensor.DownloadToArray()[0];
    }
}
}

```

A.9 Скрипт GameSession

```
public class GameSession : MonoBehaviour
{
    public static GameSession Instance { get; private set; }

    [Header("Setting")]
    [SerializeField] bool isTrackingInputs = true;

    [Header("Death Config")]
    [SerializeField] float deathWindow = 30f;
    [SerializeField] int deathClusterSize = 50;
    [SerializeField] int deathClusterMin = 2;
    [SerializeField] float deathClusterDistance = 20f;

    [Header("Idle Config")]
    [SerializeField] float idleThreshold = 2f;

    [Header("Psychological Tracking")]
    [Tooltip("How long to measure APM (Actions Per Minute) and Jitter")]
    public float behavioralWindowSize = 10f;
    [SerializeField] float jitterPanicThreshold = 50f;

    private int totalDeaths;

    [Serializable]
    public class ObstacleData
    {
        public string obstacleType;
        public string obstacleId;
        public Vector3 position;
        public int attempts = 0;
        public int successes = 0;

        public float SuccessRate => attempts > 0 ? (float)successes / attempts :
0f;
    }

    private float sessionStartTime;

    private int retries = 0;

    private float levelStartTime;

    private int currentLevel = 0;
    public int CurrentLevel => currentLevel;

    // Input tracking & APM (Actions per minute)
    private int inputCount = 0;
    private float inputWindowStart;
    [HideInInspector] public float inputWindowSize = 30f;

    // Нові змінні для Jitter та Idle
    private float totalIdleTimeThisLevel = 0f;
    private float idleSessionStart = -1f;
    private bool isPlayerIdle = false;

    private float totalCameraJitterThisLevel = 0f;
    private Vector2 lastMousePos;

    // Time to recover (Час після смерті до першого руху)
    private float lastSpawnTime = 0f;
    private float averageTimeToRecover = 0f;
```

```

private int spawnCount = 0;
private bool isRecovering = false;

public float CurrentInputRate => (Time.time - inputWindowStart) > 0.1f ?
inputCount / (Time.time - inputWindowStart) : 0f;

// Публічні гетери для нової ML системи
public float AverageTimeToRecover => spawnCount > 0 ? averageTimeToRecover /
spawnCount : 0f;
public float TotalIdleTimeThisLevel => totalIdleTimeThisLevel;
public float TotalCameraJitter => totalCameraJitterThisLevel;
public float TimeSpentOnLevel => Time.time - levelStartTime;

private Dictionary<int, int> deathsPerLevel = new();
private Dictionary<Key, float> keyDownTimes = new Dictionary<Key, float>();
private Dictionary<string, ObstacleData> obstacleStats = new
Dictionary<string, ObstacleData>();

private int deathsSinceLastCheckpoint = 0;
public int CheckpointDeaths => deathsSinceLastCheckpoint;

public struct DeathRecord
{
    public Vector3 position;
    public float timestamp;
}
private List<DeathRecord> recentDeaths = new();

public Vector3? ActiveDeathClusterCenter { get; private set; }
public int ClusterDeaths => recentDeaths.Count;

// ===== DDA METHODS =====
public IReadOnlyDictionary<int, int> DeathsPerLevel => deathsPerLevel;
public IReadOnlyDictionary<string, ObstacleData> ObstacleStats =>
obstacleStats;
public IReadOnlyList<DeathRecord> RecentDeaths => recentDeaths;
public int GetTotalDeaths() => totalDeaths;
public float GetStartTime() => sessionStartTime;
public float LevelStartTime() => levelStartTime;

private void RegisterPlayerFeedback(string label, float desiredMultiplier)
{
    Vector3 pos = ActiveDeathClusterCenter ?? Vector3.zero;
    if (pos == Vector3.zero)
    {
        var p = GameObject.FindGameObjectWithTag("Player");
        if (p != null) pos = p.transform.position;
    }

    AnalyticsManager.LogEvent("player_feedback", new
    {
        level = currentLevel,
        label = label,
        desired_multiplier = desiredMultiplier,
        deaths_at_current_checkpoint = deathsSinceLastCheckpoint,
        recent_cluster_deaths = recentDeaths.Count,

        time_spent_on_level = TimeSpentOnLevel,
        avg_time_to_recover = AverageTimeToRecover,
        total_idle_time = totalIdleTimeThisLevel,
        camera_jitter = totalCameraJitterThisLevel,
        current_input_rate = CurrentInputRate,

        position = new float[] { pos.x, pos.y, pos.z }
    });
}

```

```

    });
}

void TrackPsychologicalMetrics(Keyboard keyboard, Mouse mouse)
{
    // 1. Camera Jitter
    if (mouse != null)
    {
        Vector2 currentMousePos = mouse.position.ReadValue();
        float delta = Vector2.Distance(currentMousePos, lastMousePos);

        if (delta > jitterPanicThreshold)
        {
            totalCameraJitterThisLevel += delta * Time.deltaTime;
        }
        lastMousePos = currentMousePos;
    }

    // 2. Time to Recover
    if (isRecovering)
    {
        bool inputDetected = (keyboard != null &&
keyboard.anyKey.wasPressedThisFrame) ||
                                (mouse != null &&
(mouse.leftButton.wasPressedThisFrame || mouse.rightButton.wasPressedThisFrame
|| mouse.delta.ReadValue().sqrMagnitude > 0.01f));

        if (inputDetected)
        {
            float recoverTime = Time.time - lastSpawnTime;
            averageTimeToRecover += recoverTime;
            isRecovering = false;
        }
    }
}

// ===== METRICS =====

public void StartLevel(int level)
{
    currentLevel = level;
    levelStartTime = Time.time;

    recentDeaths.Clear();
    obstacleStats.Clear();
    ActiveDeathClusterCenter = null;
    deathsSinceLastCheckpoint = 0;

    // Скидання нових метрик
    totalIdleTimeThisLevel = 0f;
    totalCameraJitterThisLevel = 0f;
    averageTimeToRecover = 0f;
    spawnCount = 0;
    idleSessionStart = Time.time;
    isPlayerIdle = false;

    AnalyticsManager.LogEvent("level_start", new
    {
        level,
        start_time = DateTime.UtcNow.ToString("o")
    });
}

public void NotifyPlayerSpawned()

```

```

{
    lastSpawnTime = Time.time;
    spawnCount++;
    isRecovering = true;
}

public void EndLevel()
{
    if (isPlayerIdle)
    {
        float actualIdleDuration = Time.time - (idleSessionStart +
idleThreshold);
        if (actualIdleDuration > 0) totalIdleTimeThisLevel +=
actualIdleDuration;
        isPlayerIdle = false;
    }

    float timeOnLevel = Time.time - levelStartTime;
    AnalyticsManager.LogEvent("level_end", new
    {
        level = currentLevel,
        duration = timeOnLevel,
        total_idle = totalIdleTimeThisLevel,
        total_jitter = totalCameraJitterThisLevel,
        end_time = DateTime.UtcNow.ToString("o")
    });
}

public void RegisterCheckpoint(int checkpointId, Vector3 pos)
{
    AnalyticsManager.LogEvent("checkpoint", new
    {
        level = currentLevel,
        checkpoint_id = checkpointId,
        position = new float[] {pos.x, pos.y, pos.z}
    });
    GameEvents.OnCheckpointReached?.Invoke(pos);

    if (deathsSinceLastCheckpoint == 0)
    GameEvents.OnPlayerDominating?.Invoke(pos);
    deathsSinceLastCheckpoint = 0;
}

public void RegisterDeath(Vector3 pos, string cause)
{
    deathsSinceLastCheckpoint++;

    if (!deathsPerLevel.ContainsKey(currentLevel))
    deathsPerLevel[currentLevel] = 0;

    deathsPerLevel[currentLevel]++;
    totalDeaths++;
    AnalyticsManager.LogEvent("death", new
    {
        death_id = deathsPerLevel[currentLevel],
        level = currentLevel,
        cause = cause,
        time_spent_on_level = Time.time - levelStartTime,
        total_idle_time = totalIdleTimeThisLevel,
        camera_jitter = totalCameraJitterThisLevel,
        input_rate = CurrentInputRate,
        avg_time_to_recover = AverageTimeToRecover,
        position = new float[] { pos.x, pos.y, pos.z }
    });
}

```

```

GameEvents.OnPlayerDeath?.Invoke(pos, cause);

recentDeaths.RemoveAll(d => Time.time - d.timestamp > deathWindow);
recentDeaths.Add(new DeathRecord { position = pos, timestamp = Time.time
});

AnalyticsManager.LogEvent("deaths_per_window", new
{
    level = currentLevel,
    window_size = deathWindow,
    count = recentDeaths.Count
});

if (recentDeaths.Count > deathClusterSize)
    recentDeaths.RemoveAt(0);

CheckErrorClusters();
}

public void RegisterRetry(float secondsSinceDeath)
{
    retries++;
    AnalyticsManager.LogEvent("retry", new
    {
        level = currentLevel,
        time_since_death = secondsSinceDeath
    });
}

public void RegisterObstacleAttemp(string obstacleType, string obstacleId,
Vector3 pos, bool success)
{
    if (!obstacleStats.ContainsKey(obstacleId))
        obstacleStats[obstacleId] = new ObstacleData
        {
            obstacleType = obstacleType,
            obstacleId = obstacleId,
            position = pos
        };

    var stats = obstacleStats[obstacleId];
    stats.attempts++;

    if (success) stats.successes++;
    obstacleStats[obstacleId] = stats;

    AnalyticsManager.LogEvent("obstacle_result", new
    {
        obstacle_type = obstacleType,
        obstacle_id = obstacleId,
        position = new float []{ pos.x, pos.y, pos.z},
        success = success,
        attempts = stats.attempts,
        successes = stats.successes,
        success_rate = stats.SuccessRate
    });
}

// ===== INTERNAL METHODS =====

void CheckErrorClusters()
{
    if (recentDeaths.Count < deathClusterMin)

```

```

    {
        ActiveDeathClusterCenter = null;
        return;
    }

    Vector3 last = recentDeaths[^1].position;
    int clusterCount = 0;
    Vector3 clusterSum = Vector3.zero;

    foreach (var record in recentDeaths)
    {
        if (Vector3.Distance(last, record.position) < deathClusterDistance)
        {
            clusterCount++;
            clusterSum += record.position;
        }
    }

    if (clusterCount >= deathClusterMin)
    {
        Vector3 clusterCenter = clusterSum / clusterCount;
        ActiveDeathClusterCenter = clusterCenter;

        GameEvents.OnDeathClusterDetected?.Invoke(clusterCenter);

        AnalyticsManager.LogEvent("death_cluster", new
        {
            level = currentLevel,
            position = new float[] { last.x, last.y, last.z },
            deaths_in_cluster = clusterCount
        });
    }
    else
    {
        ActiveDeathClusterCenter = null;
    }
}

void CheckIdle(Keyboard keyboard, Mouse mouse)
{
    bool hasInput = false;

    if (keyboard != null && keyboard.anyKey.isPressed) hasInput = true;
    if (mouse != null && (mouse.leftButton.isPressed ||
mouse.rightButton.isPressed || mouse.delta.ReadValue().sqrMagnitude > 0.01f))
hasInput = true;

    if (hasInput)
    {
        if (isPlayerIdle)
        {
            float actualIdleDuration = Time.time - (idleSessionStart +
idleThreshold);
            if (actualIdleDuration < 0) actualIdleDuration = 0;

            totalIdleTimeThisLevel += actualIdleDuration;

            isPlayerIdle = false;
            AnalyticsManager.LogEvent("idle_ended", new { duration =
actualIdleDuration });
        }
        idleSessionStart = Time.time;
    }
    else

```

```

        {
            if (!isPlayerIdle && (Time.time - idleSessionStart > idleThreshold))
            {
                isPlayerIdle = true;
                AnalyticsManager.LogEvent("idle_started", new { level =
currentLevel });
            }
        }
    }
}

```

A.10 Скрипт TensionAtmosphere

```

public class TensionAtmosphere : MonoBehaviour, IDynamicAtmosphere
{
    [Header("DDA Audio Settings")]
    [SerializeField, Range(0f, 2f)] protected float ddaWeight = 1.0f;

    [Header("Lighting Settings")]
    [SerializeField] private Light directionalLight;
    [SerializeField] private Color relaxedColor = new Color(0.9f, 0.9f, 1f); //
Холодний, спокійний
    [SerializeField] private Color stressedColor = new Color(0.4f, 0.1f, 0.1f);
    // Тривожний темно-червоний

    [Header("Fog Settings")]
    [SerializeField] private float baseFogDensity = 0.02f;
    [SerializeField] private float maxFogDensity = 0.08f;

    [Header("Post-Processing (Global Volume)")]
    [SerializeField] private Volume globalVolume;
    [SerializeField] private float maxVignetteIntensity = 0.5f;
    [SerializeField] private float maxChromaticAberration = 1.0f;

    [Header("Lerp Speed")]
    [SerializeField] private float transitionSpeed = 1.5f;

    private float currentTensionTarget = 1f;

    // Кешовані ефекти
    private Vignette vignette;
    private ChromaticAberration chromaticAberration;
}

void OnEnable()
{
    GameEvents.OnGlobalTensionChanged += HandleGlobalTension;
}

void OnDisable()
{
    GameEvents.OnGlobalTensionChanged -= HandleGlobalTension;
}

private void HandleGlobalTension(float predictedMultiplier)
{
    AdjustAtmosphere(predictedMultiplier, AdjustmentMode.Immediate);
}

public void AdjustAtmosphere(float multiplier, AdjustmentMode mode =
AdjustmentMode.Immediate)
{
    // Інвертуємо: якщо multiplier = 0.5 (дуже важко), напруга = 2.0
    if (Mathf.Approximately(ddaWeight, 0f)) return;
}

```

```

        float weightedMultiplier = Mathf.Lerp(1f, multiplier, ddaWeight);
        currentTensionTarget = Mathf.Clamp(1f / weightedMultiplier, 0.5f, 2.0f);
    }

    void Update()
    {
        float lerpFactor = Mathf.InverseLerp(0.5f, 2.0f, currentTensionTarget);

        // 1. Світло
        if (directionalLight != null)
        {
            Color targetColor = Color.Lerp(relaxedColor, stressedColor,
            lerpFactor);
            directionalLight.color = Color.Lerp(directionalLight.color,
            targetColor, Time.deltaTime * transitionSpeed);
        }

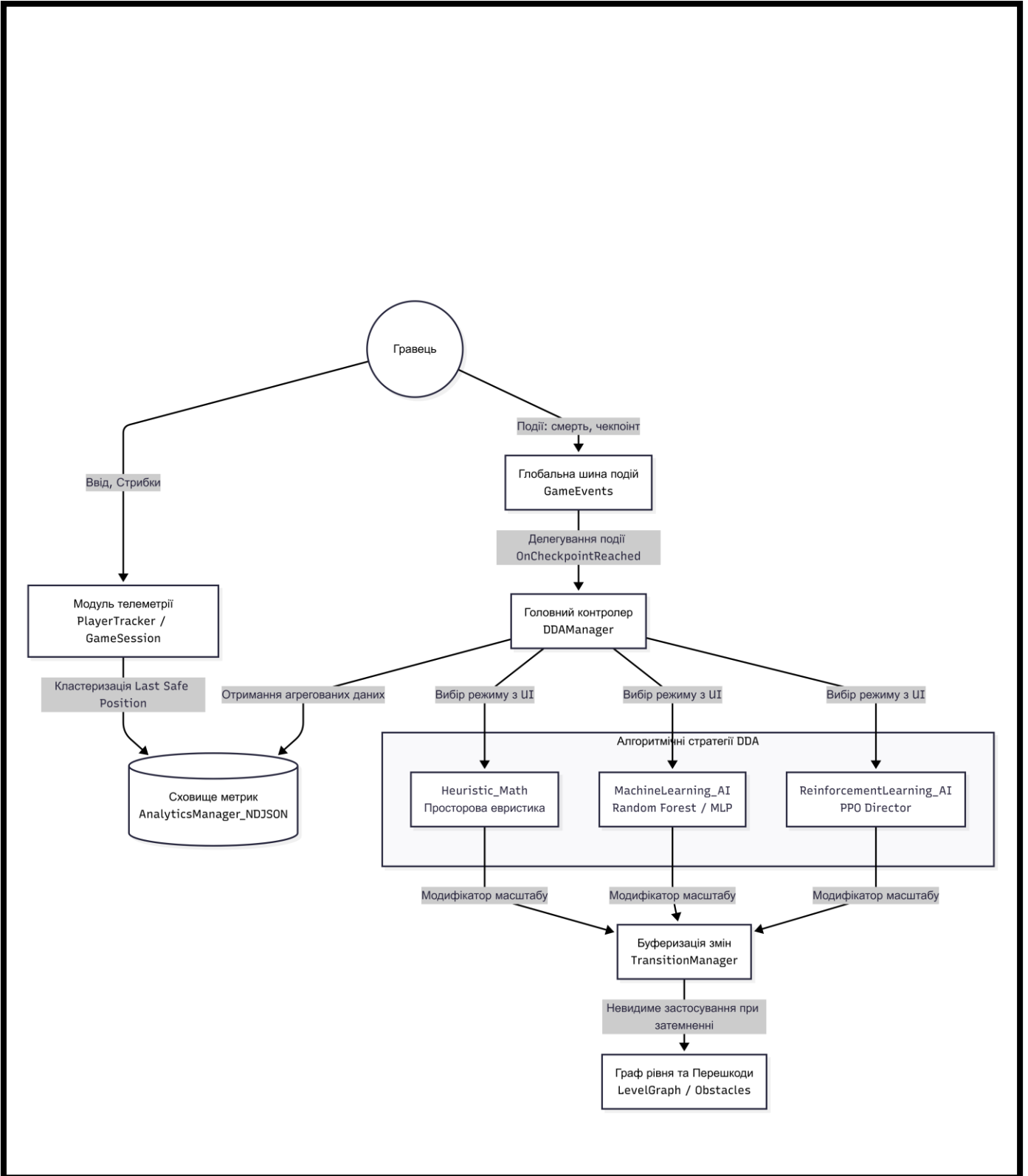
        // 2. Туман
        float targetFog = Mathf.Clamp(baseFogDensity * currentTensionTarget,
        baseFogDensity, maxFogDensity);
        RenderSettings.fogDensity = Mathf.Lerp(RenderSettings.fogDensity,
        targetFog, Time.deltaTime * transitionSpeed);

        // 3. Post-Processing (Тільки якщо є ефекти)
        if (vignette != null)
        {
            vignette.intensity.value = Mathf.Lerp(vignette.intensity.value,
            maxVignetteIntensity * lerpFactor, Time.deltaTime * transitionSpeed);
        }

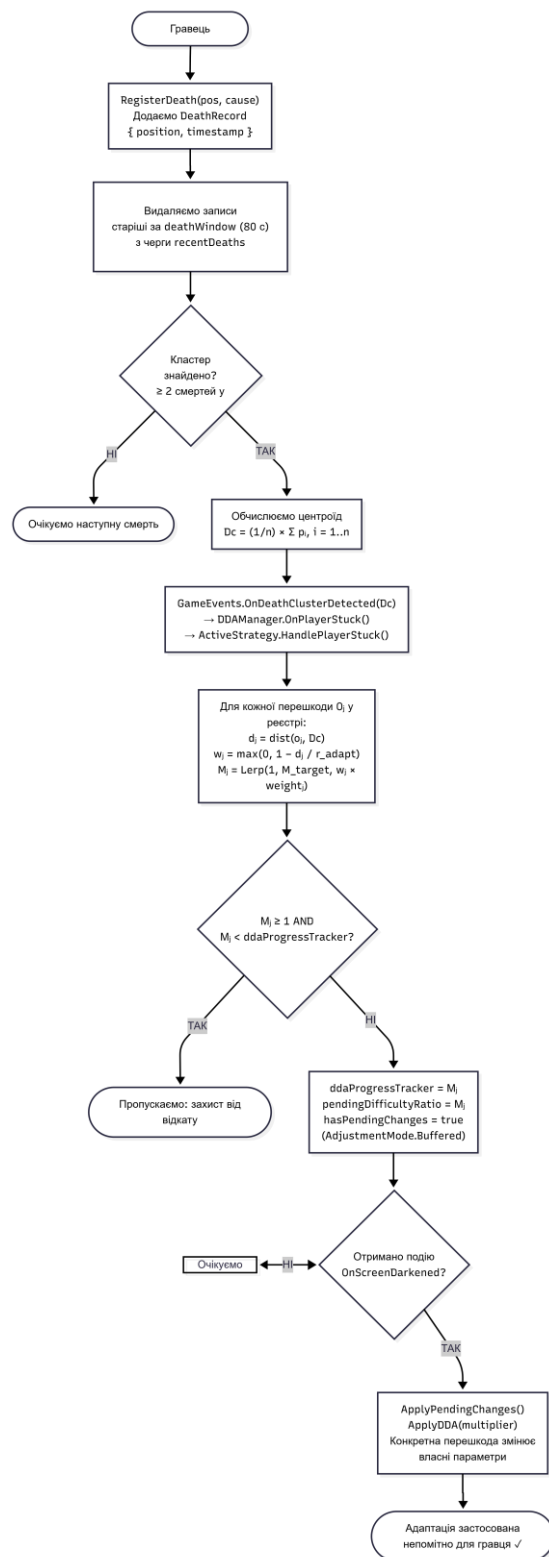
        if (chromaticAberration != null)
        {
            // Збільшуємо спотворення при стресі
            chromaticAberration.intensity.value =
            Mathf.Lerp(chromaticAberration.intensity.value, maxChromaticAberration *
            lerpFactor, Time.deltaTime * transitionSpeed);
        }
    }
}

```

ДОДАТОК Б
Графічні матеріали



					Бакалаврська кваліфікаційна робота			
					Функціональна схема системи			
Змн.	Арк.	Прізвище	Підпис	Дата				
	Розроб.	Верещак Б. О.						
	Керівник	Патерера Ю. І.						
	Н. Контр.							
	Зав. каф.	Лобур М.В.						
						Літера	Маса	Масштаб
						Н		
						Аркуш 1		Аркушів 6
						НУ «ЛП», ІКНІ, каф. САП, гр. ПП-44		



Бакалаврська кваліфікаційна робота

Функціональна схема застосування адаптації

Літера Маса Масштаб

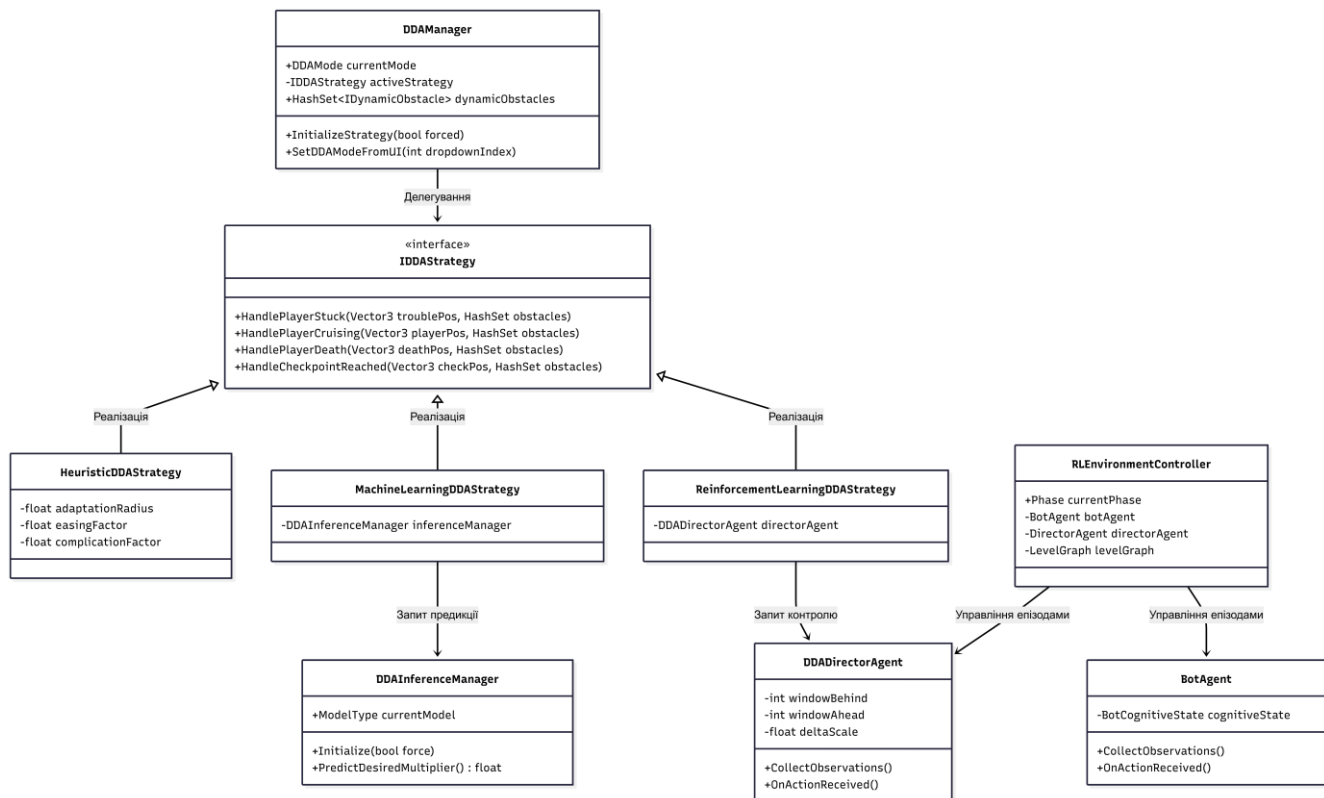
Н

Аркуш 2

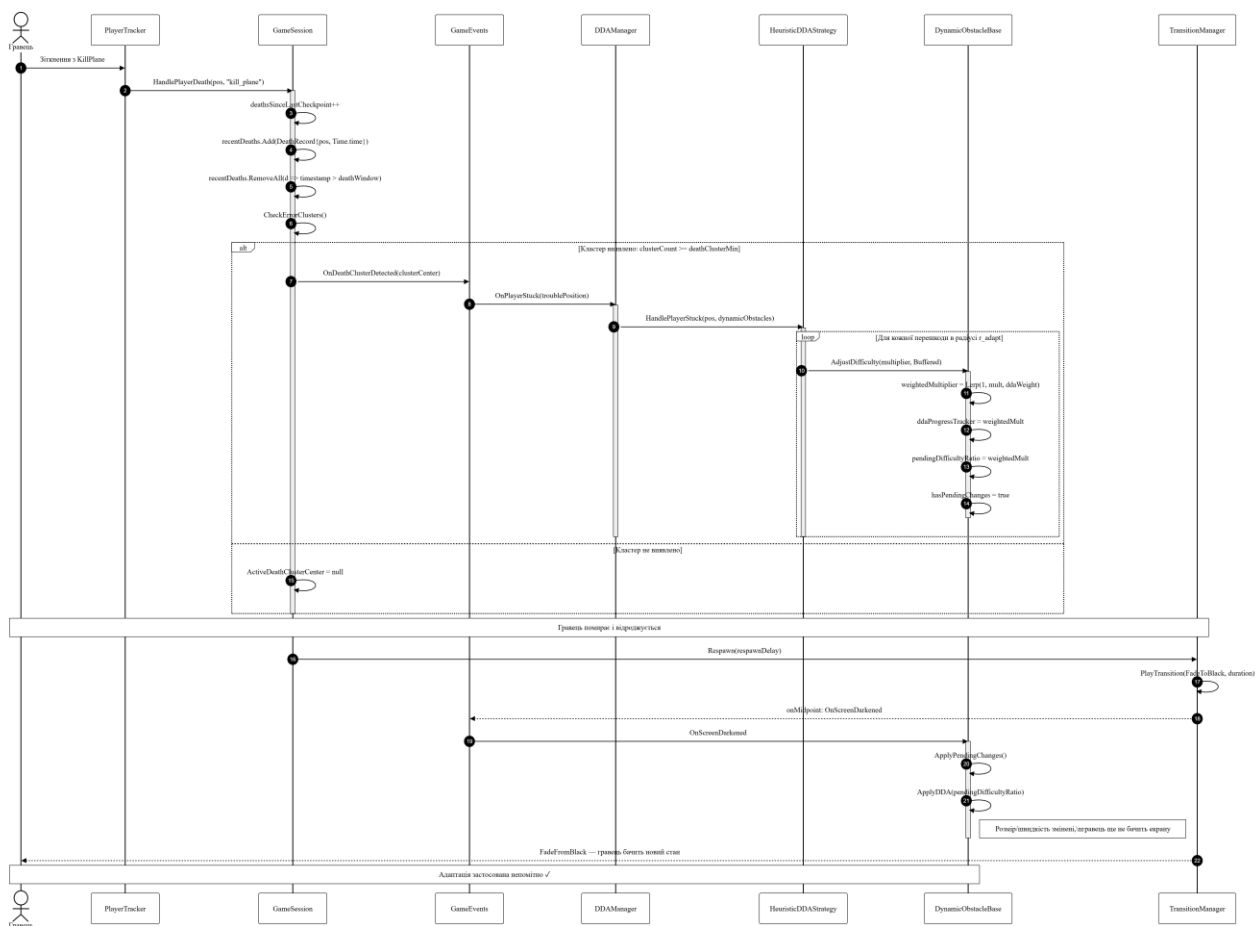
Аркушів 6

НУ «ЛП», ІКНІ,
каф. САП, гр. ПП-44

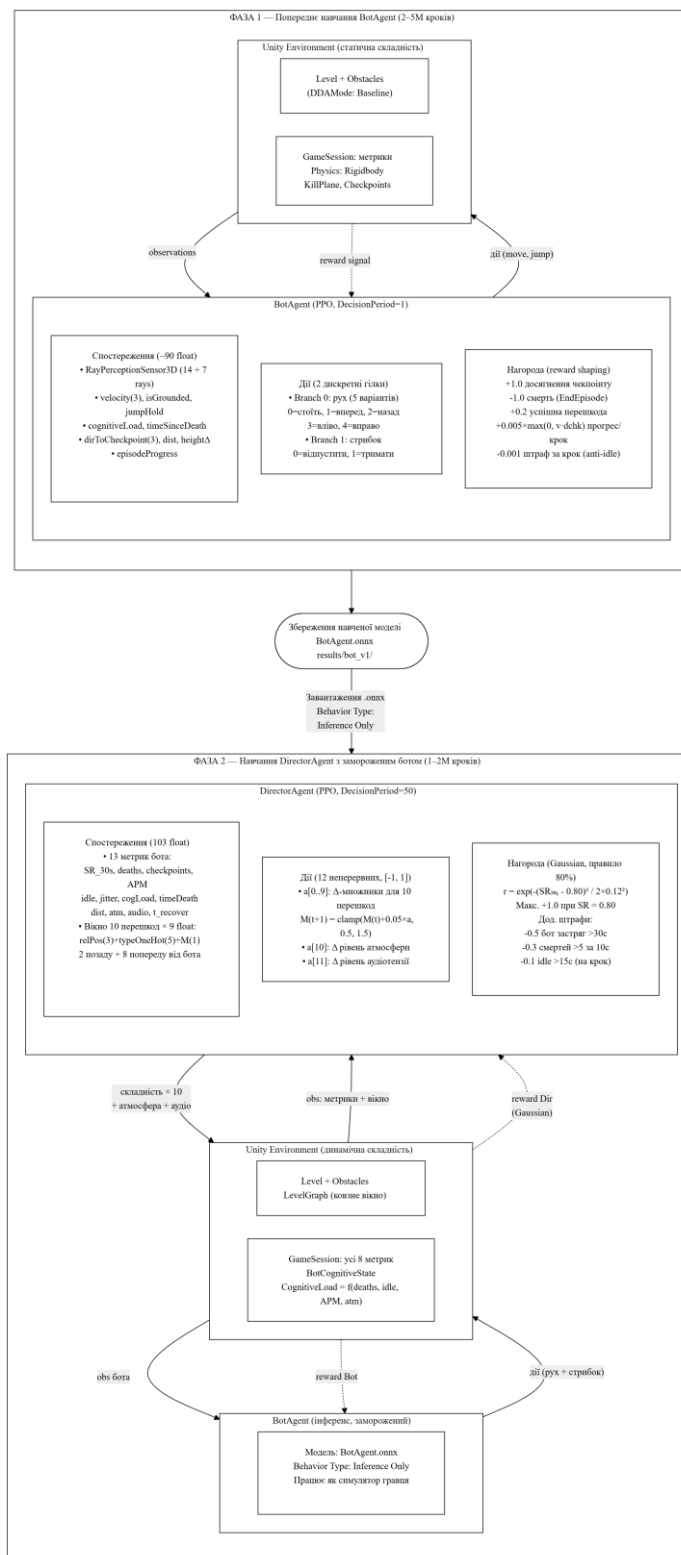
Змн.	Арк.	Прізвище	Підпис	Дата
Розроб.		Верещак Б. О.		
Керівник		Патерера Ю. І.		
Н. Контр.				
Зав. каф.		Лобур М.В.		



					Бакалаврська кваліфікаційна робота					
					Структурна схема динамічної адаптації складності	Літера		Маса	Масштаб	
							N			
						Аркуш 3		Аркушів 6		
						НУ «ЛП», ІКНІ, каф. САП, гр. ПП-44				
Змн.	Арк.	Прізвище	Підпис	Дата						
	Розроб.	Верещак Б. О.								
	Керівник	Патерега Ю. І.								
	Н. Контр.									
	Зав. каф.	Лобур М.В.								



					Бакалаврська кваліфікаційна робота					
					Діаграма потоків даних системи	Літера		Маса	Масштаб	
Змн.	Арк.	Прізвище	Підпис	Дата						
	Розроб.	Верещак Б. О.								
	Керівник	Патерега Ю. І.								



Бакалаврська кваліфікаційна робота

Блок-схема конвєса машинного навчання

Літера		Маса	Масштаб
Н			
Аркуш 6		Аркушів 6	

Змн.	Арк.	Прізвище	Підпис	Дата
Розроб.		Верещак Б. О.		
Керівник		Патерера Ю. І.		
Н. Контр.				
Зав. каф.		Лобур М.В.		

НУ «ЛП», ІКНІ,
каф. САП, гр. ПП-44